

Fondamenti di Analisi dei Dati

from data analysis to predictive techniques

Prof. Antonino Furnari (antonino.furnari@unict.it)
Corso di Studi in Informatica
Dip. di Matematica e Informatica
Università di Catania

Classification and K-Nearest Neighbours

Measuring a classifier, the simplest one that works, and what goes wrong in high dimensions

Classification Task Definition

Mathematical Framework

A classification model is a function

$$h : \mathbb{R}^n \to \{0, \ldots, M-1\}$$

that maps numerical input vectors to M discrete class labels.

Given training data $D = \{(x_i, y_i)\}_{i=1}^{N}$, we minimise empirical risk using a loss function that **counts incorrect predictions**. For example:

$$L(\hat{y}, y) = \begin{cases} 1 & \text{if } \hat{y} \neq y \\ 0 & \text{if } \hat{y} = y \end{cases}$$

Empirical Risk in this case:

$$R_{emp}(h) = \frac{1}{N} \sum_{i=1}^{N} L(\hat{y}_i, y_i) = \frac{\text{number of incorrect predictions}}{N}$$

Key Applications

- Spam email detection (binary classification)
- Social media content categorization
- Object recognition in images (multi-class)
- Medical diagnosis and screening

Evaluation Measures: Accuracy and Error Rate

Evaluation measures guide model training by providing feedback on performance, help tune hyperparameters, and ultimately assess how well an algorithm generalises to unseen data.

$$Y_{TE} = \left\{ y^{(i)} \mid \left(\mathbf{x}^{(i)}, y^{(i)} \right) \in TE \right\}_i \qquad \hat{Y}_{TE} = \left\{ h\left(\mathbf{x}^{(i)} \right) \mid \left(\mathbf{x}^{(i)}, y^{(i)} \right) \in TE \right\}_i$$

Accuracy

The percentage of test examples for which the algorithm predicted the correct label.

$$Accuracy(Y_{TE}, \hat{Y}_{TE}) = \frac{\left| \left\{ y^{(i)} : y^{(i)} = \hat{y}^{(i)} \right\} \right|}{|Y_{TE}|}$$

If a model correctly predicts 70 out of 100 test examples, its accuracy is 0.7 or 70%.

Error Rate

The proportion of incorrect predictions — the complement of correctness.

$$ErrorRate(Y_{TE}, \hat{Y}_{TE}) = \frac{\left| \left\{ y^{(i)} : y^{(i)} \neq \hat{y}^{(i)} \right\} \right|}{|Y_{TE}|} = 1 - Accuracy$$

An accuracy of 70% implies an error rate of 30%.

The challenge of imbalanced datasets. Consider 10 000 entries: 500 from Class 0 and 9 500 from Class 1. A naive classifier that always predicts "Class 1" achieves **95% accuracy** — and is practically useless, since it never identifies a single instance of Class 0. **Accuracy alone is misleading when the classes are imbalanced.**

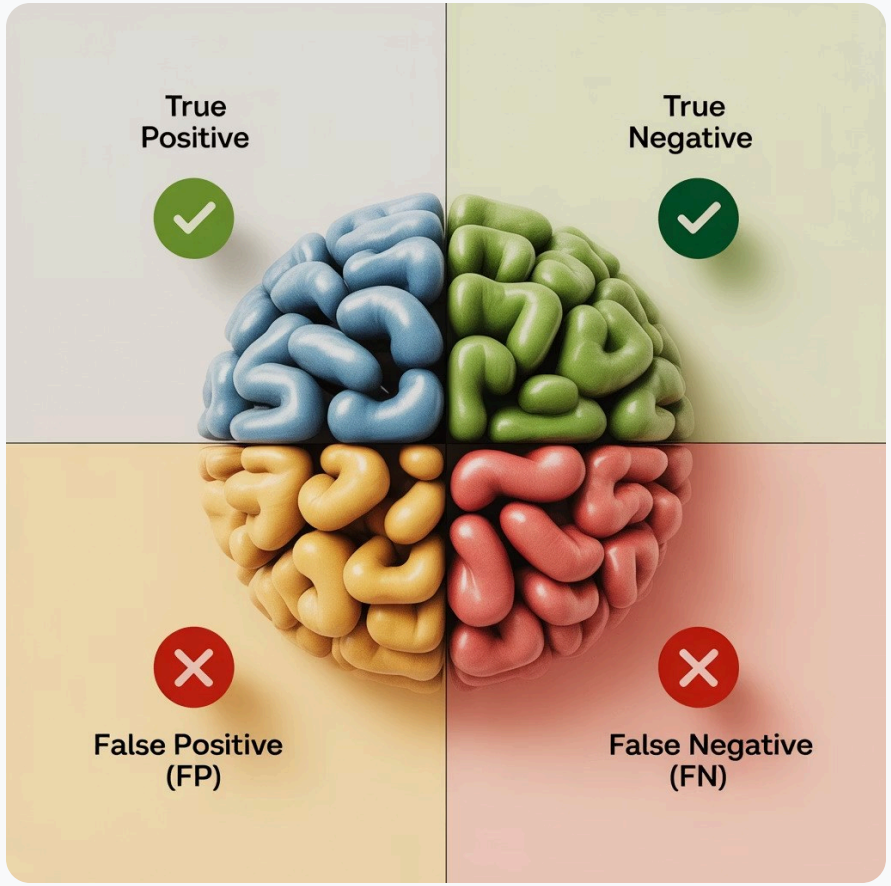
Understanding Error Types

Classification errors fall into two categories that reveal different failure modes:

- **False Positives (Type 1):** Classifying a negative example as positive
- **False Negatives (Type 2):** Classifying a positive example as negative

Correct predictions include:

- **True Positives:** Correctly identified positive examples
- **True Negatives:** Correctly identified negative examples



The Confusion Matrix

The confusion matrix provides a **complete diagnostic view** of classifier performance, showing exactly where predictions succeed and fail.

Confusion Matrix

CONFUSION MATRIX		PREDICTED LABELS	
		POSITIVE	NEGATIVE
TRUE LABELS	POSITIVE	TP	FN
	NEGATIVE	FP	TN

A strong classifier has large numbers on the diagonal (TP and TN) and small numbers off-diagonal (FP and FN).

$$Accuracy = \frac{TP + TN}{TP + FN + FP + TN}$$

Spam Detector Example

CONFUSION MATRIX		PREDICTED LABELS	
		SPAM	LEGITIMATE
TRUE LABELS	SPAM	40	10
	LEGITIMATE	20	30

Accuracy: 70%. The matrix reveals **20 legitimate emails wrongly flagged as spam**.

$$Accuracy = \frac{40 + 30}{40 + 10 + 20 + 30} = \frac{70}{100} = 0.7$$

Imbalanced Dataset

CONFUSION MATRIX		PREDICTED LABELS	
		POSITIVE	NEGATIVE
TRUE LABELS	POSITIVE	9500	0
	NEGATIVE	500	0

$$Accuracy = \frac{9500}{9500 + 500} = 0.95$$

Even if we get a good accuracy, a glance at the confusion matrix reveals that **predictions are imbalanced**: the classifier never predicts the negative class at all.

Precision and Recall

- **Precision** measures how many of the examples which have been classified as positives were actually positives
- **Recall** measures how many of the examples which are positives have been correctly classified as positives

CONFUSION MATRIX		PREDICTED LABELS	
		POSITIVE	NEGATIVE
TRUE LABELS	POSITIVE	TP	FN
	NEGATIVE	FP	TN

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Precision reads **down the predicted-positive column**; recall reads **across the true-positive row**.

Precision vs Recall Trade-offs

When to prioritise precision

Spam filter: if an email is marked spam, it must truly be spam. False positives (blocking legitimate emails) are unacceptable.

Accept lower recall — some spam getting through is tolerable if we never block important messages.

When to prioritise recall

Medical screening: if a patient has a disease, we must detect it. False negatives (missing sick patients) are dangerous.

Accept lower precision — false positives can be caught by follow-up tests, but missing a diagnosis is catastrophic.

Spam Detector Example

CONFUSION MATRIX		PREDICTED LABELS	
		SPAM	LEGITIMATE
TRUE LABELS	SPAM	40	10
	LEGITIMATE	20	30

We can compute the following precision and recall values:

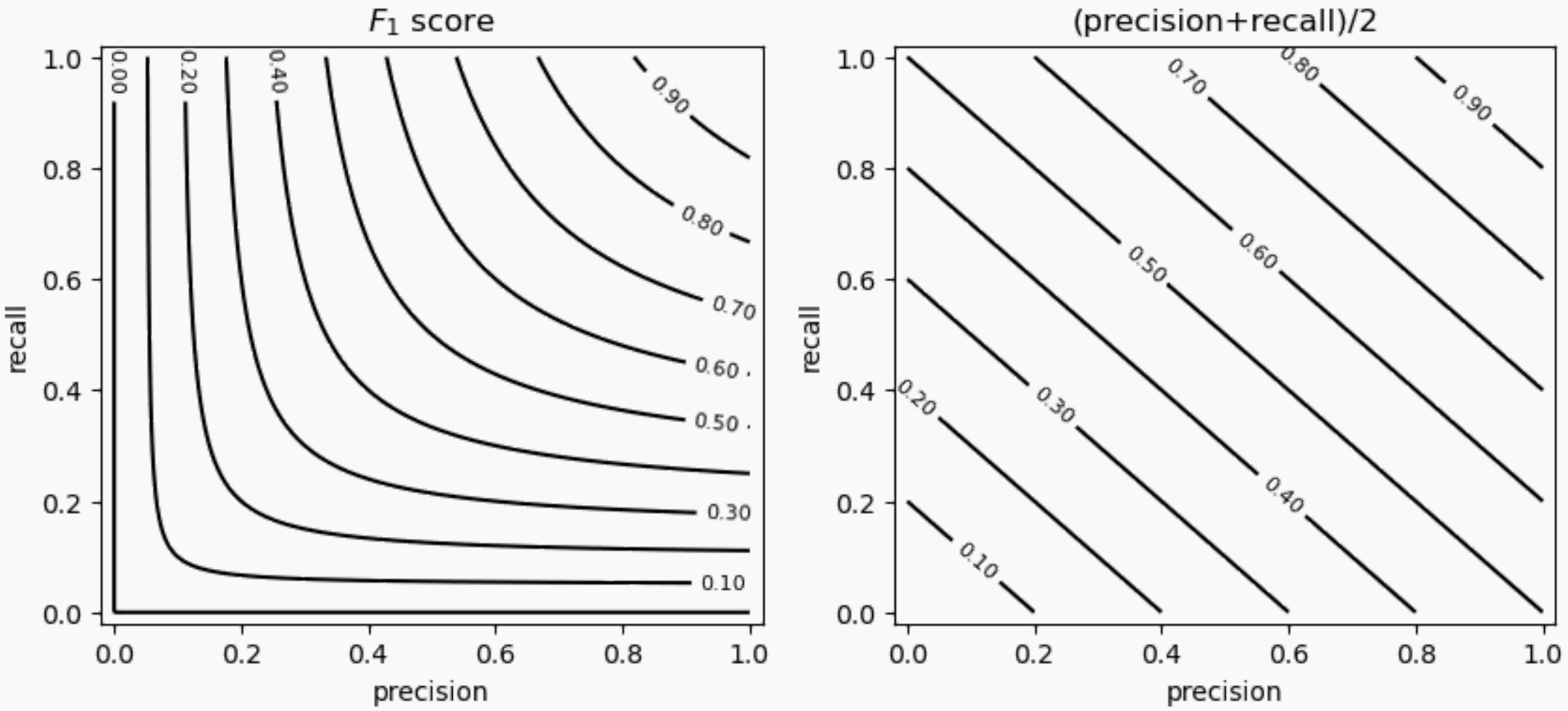
- $Precision = \frac{TP}{TP+FP} = \frac{40}{40+20} = 0.67$
- $Recall = \frac{TP}{TP+FN} = \frac{40}{40+10} = 0.8$

F_1 Score: Balancing Precision and Recall

The F_1 score provides a single metric that requires **both** high precision and high recall, through the harmonic mean:

$$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

The harmonic mean penalises imbalanced scores. To achieve a high F_1, both precision and recall must be strong — **you cannot "cheat" by maximising just one**.



Spam Detector Example: F_1

CONFUSION MATRIX		PREDICTED LABELS	
		SPAM	LEGITIMATE
TRUE LABELS	SPAM	40	10
	LEGITIMATE	20	30

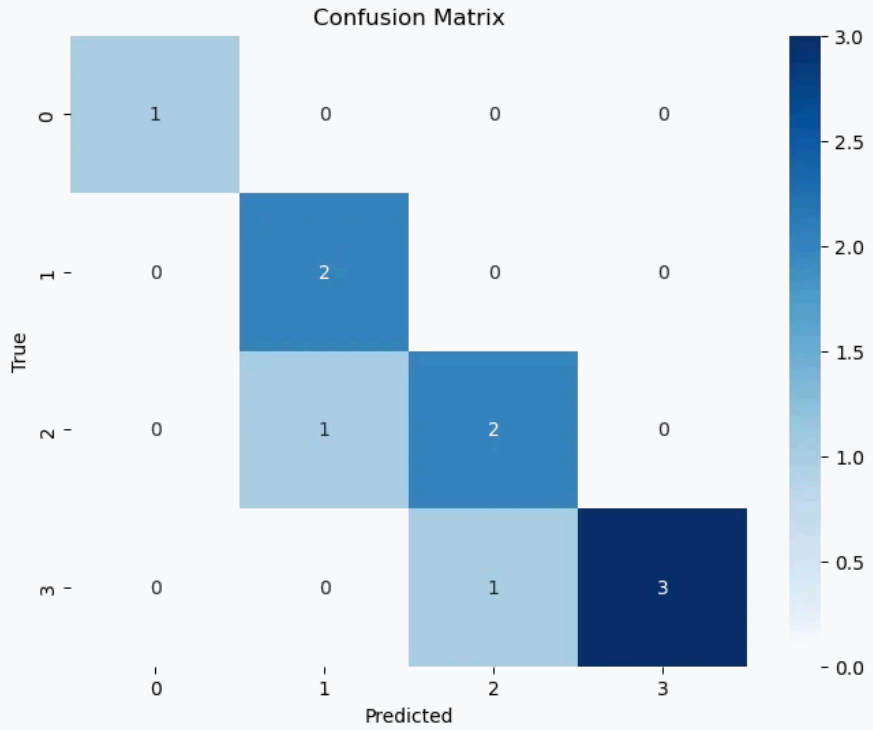
We can compute the following precision and recall values:

- $Precision = \frac{TP}{TP+FP} = \frac{40}{40+20} = 0.67$
- $Recall = \frac{TP}{TP+FN} = \frac{40}{40+10} = 0.8$

$$F_1 = 2\frac{precision \cdot recall}{precision + recall} = \frac{1.072}{1.47} = 0.72$$

The harmonic mean is symmetric, so swapping the two would not have changed F_1 — which is exactly why it hides the mistake.

Multi-Class Confusion Matrices



The confusion matrix generalises to M classes as an $M \times M$ grid where element C_{ij} shows samples of class i classified as class j.

Strong diagonal values indicate correct classifications. Off-diagonal values reveal **specific confusion patterns** between classes.

For each class, compute separate precision, recall and F_1 scores by treating it as a binary problem (class vs. all others).

Macro, Weighted and Micro Averaging

A per-class report gives precision and recall for each class. To get **one** overall number we have to average them, and there are three ways:

Macro averaging

The metric for each class independently, then the simple unweighted average.

Every **class** gets an equal vote, regardless of its size. Use it when you care about performance on rare classes.

Weighted averaging

The metric for each class, averaged by the number of true samples of each class — its support.

Every **sample** gets an equal vote. Large classes dominate.

Micro averaging

Pool the TP, FP and FN counts across all classes, then compute the metric once.

In single-label multiclass problems this equals **accuracy**.

- A macro-F_1 far below the weighted one is the signature of a model that does well on the common classes and badly on the rare ones — which the single accuracy number would have hidden.

Interpreting ROC Curves

ROC (Receiver Operating Characteristic) curves evaluate binary classifiers that output a probability or confidence score, **independently of the chosen classification threshold**.

A classifier uses a confidence function $c(\mathbf{x})$, which assigns a score to each input $\mathbf{x}$. The final classification compares this score against a threshold θ:

$$h_\theta(\mathbf{x}) = [c(\mathbf{x}) \geq \theta]$$

As the threshold θ varies, so do the counts of TP_θ, TN_θ, FP_θ and FN_θ.

Key metrics: True Positive Rate and False Positive Rate

The **True Positive Rate**, also known as Recall or Sensitivity, measures the proportion of actual positive cases correctly identified:

$$TPR_\theta = \frac{TP_\theta}{TP_\theta + FN_\theta}$$

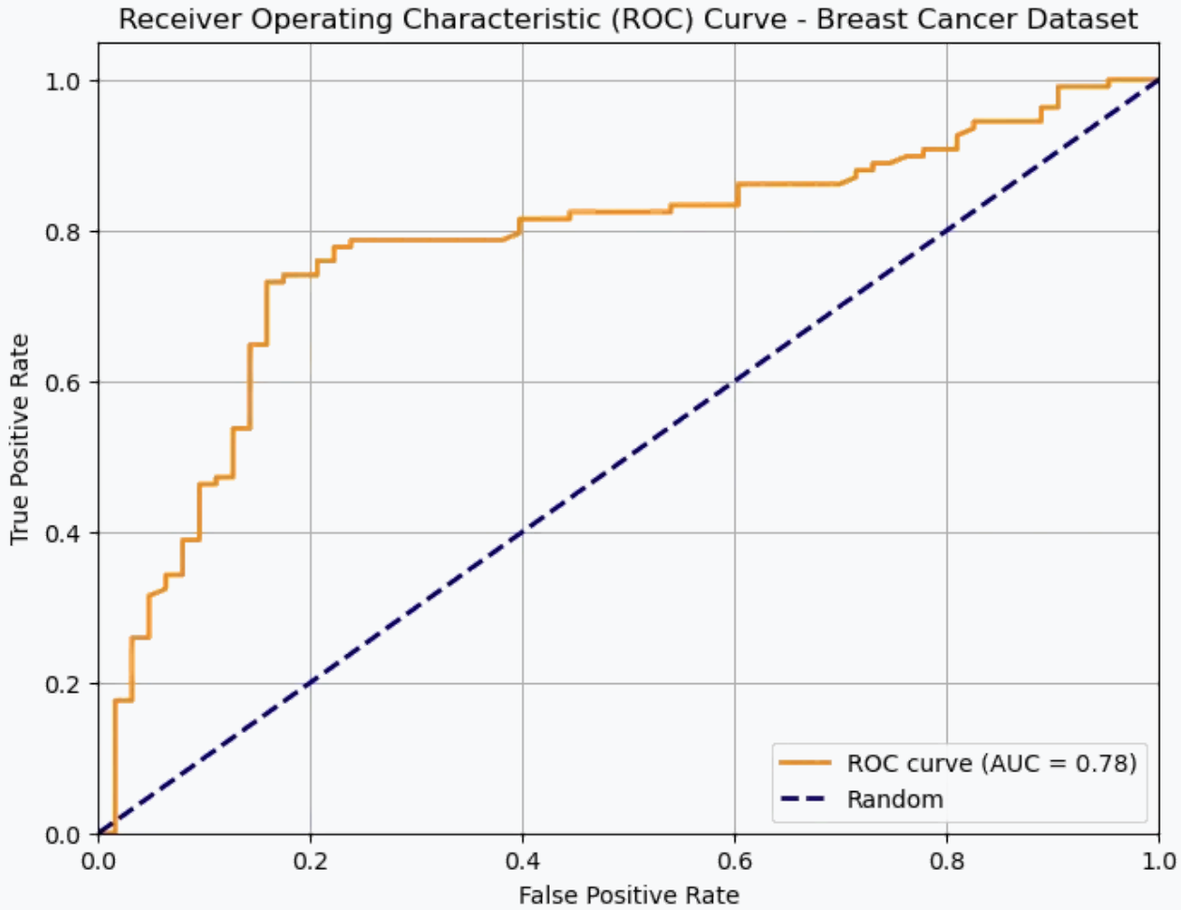
The **False Positive Rate** measures the proportion of actual negative cases incorrectly identified as positive:

$$FPR_\theta = \frac{FP_\theta}{FP_\theta + TN_\theta}$$

- **Low thresholds:** if θ is very low, nearly all instances are classified as positive. Both TPR and FPR approach 1.
- **High thresholds:** if θ is very high, nearly all instances are classified as negative. Both TPR and FPR approach 0.

Interpreting ROC Curves — cont.

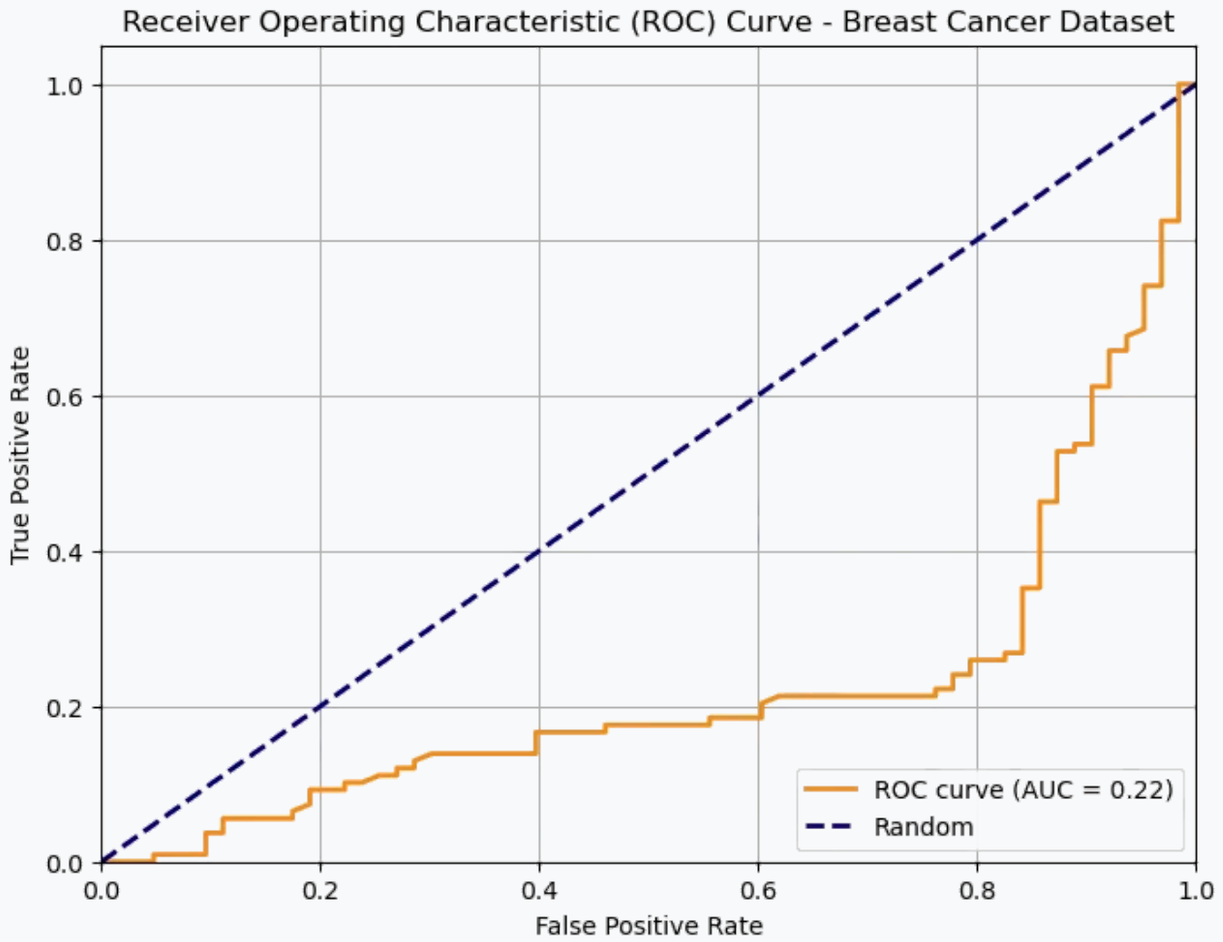
The ROC curve is created by plotting the TPR against the FPR **for every possible threshold** θ.



A good classifier's ROC curve rises quickly towards the **top-left corner** (high TPR, low FPR), indicating better separation between classes across thresholds.

The **Area Under the Curve (AUC)** summarises the curve into a single value: the probability that the classifier ranks a randomly chosen positive instance higher than a randomly chosen negative one. A higher AUC indicates better overall performance.

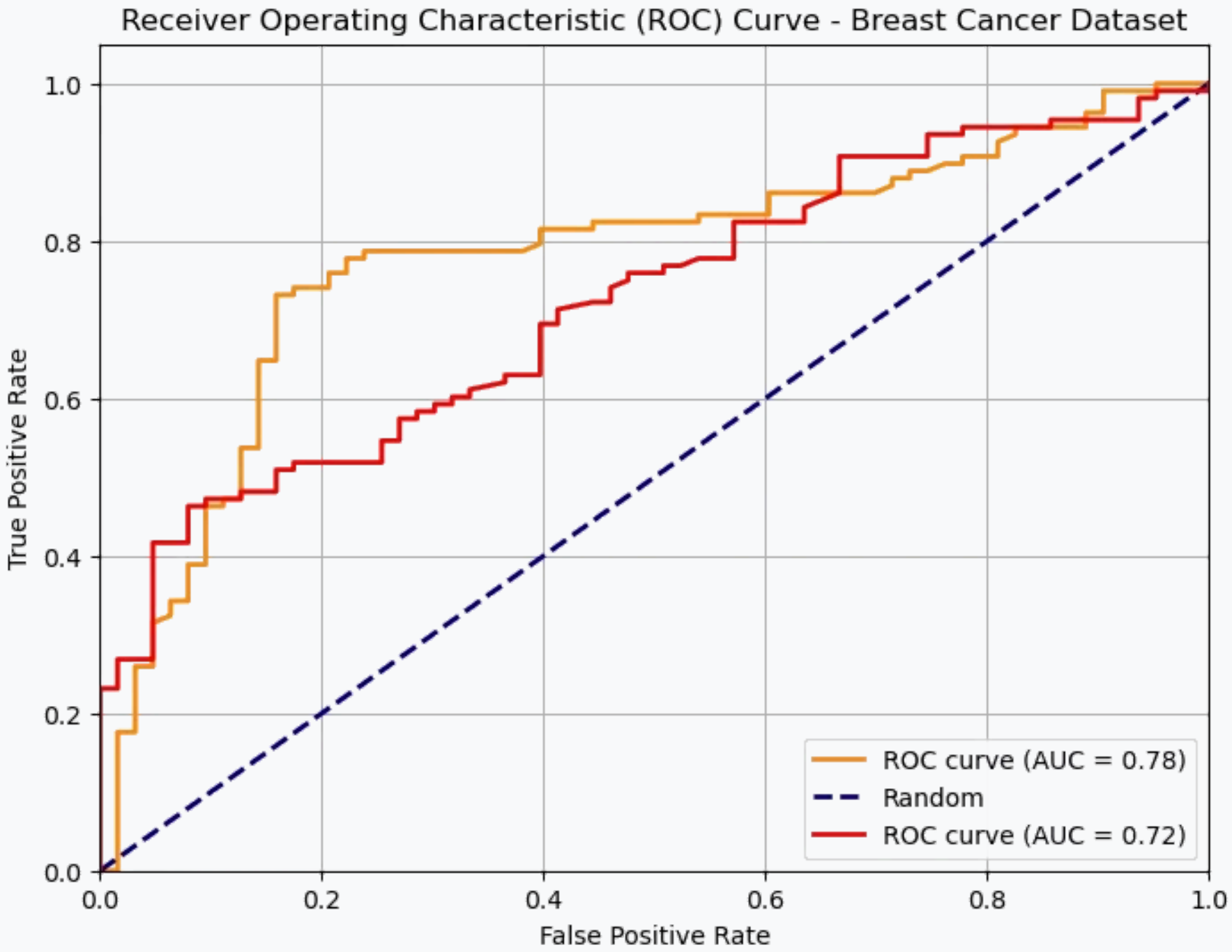
ROC Curves — a Notable Case



Any curve which is **systematically below** the diagonal indicates a classifier which is thresholding the wrong way round — we should invert the sign of the thresholding.

This is the curve of the same classifier as before, with the sign inverted: AUC 0.22 instead of 0.78.

ROC Curve to Compare Models

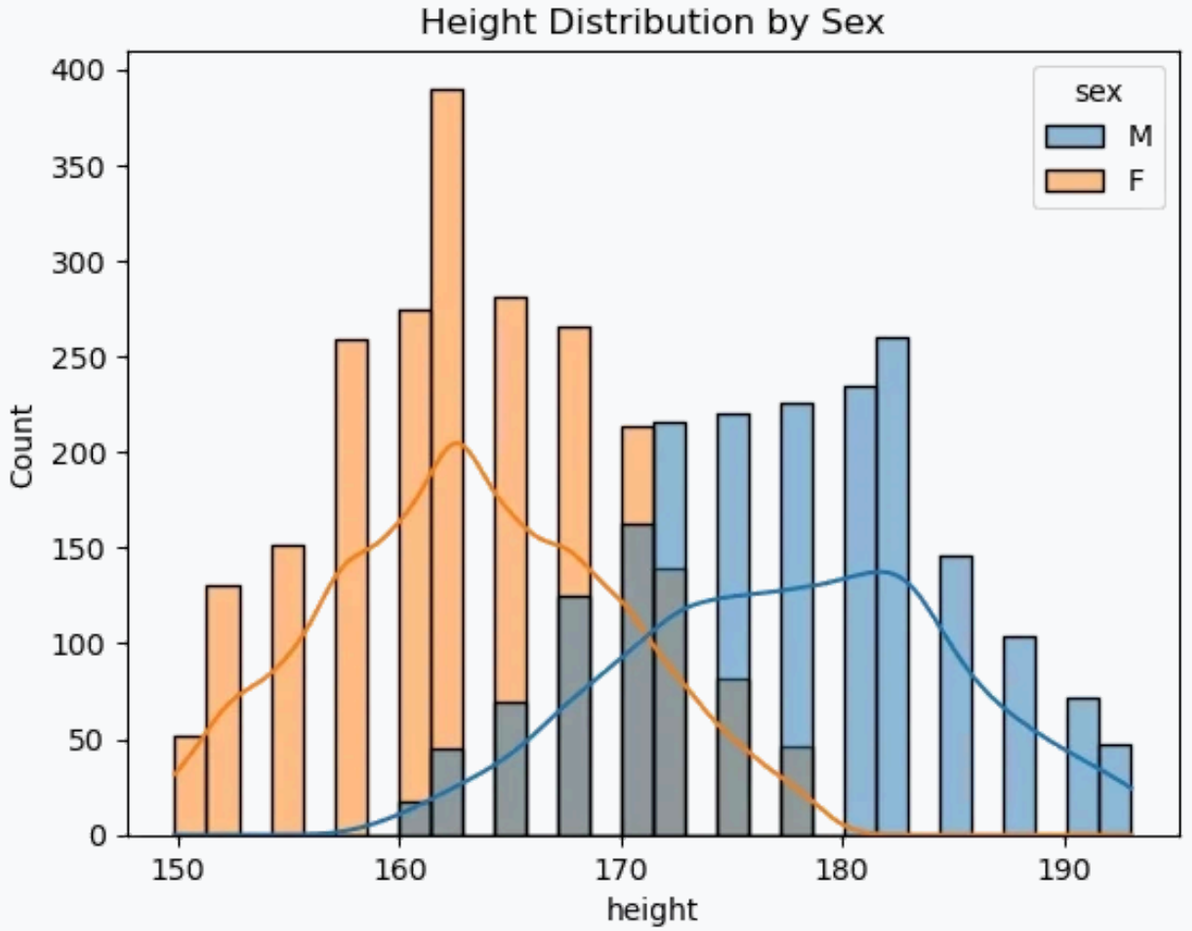


Two classifiers on the same axes: the orange curve dominates the red one over most of the range, and its AUC is higher (0.78 against 0.72). **Comparing AUCs compares classifiers without committing to a threshold.**

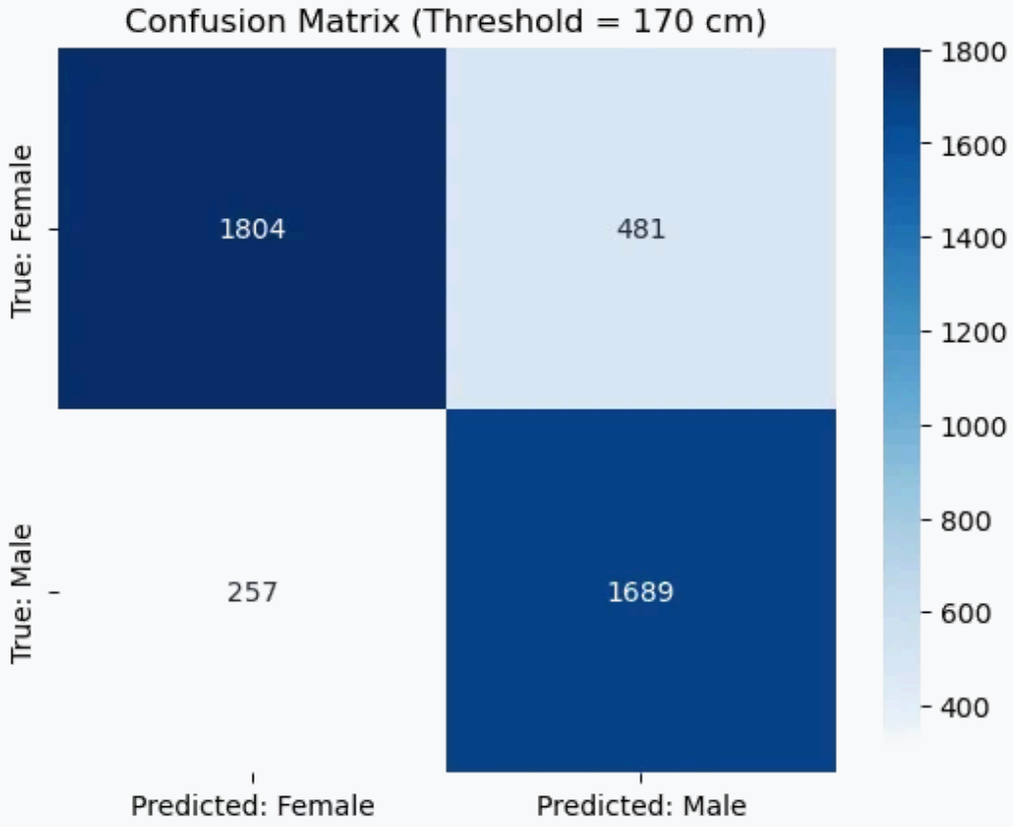
Threshold Classification Example

A simple height-based classifier demonstrates the core evaluation concepts. Using the heights-and-weights dataset, we predict biological sex with a single threshold.

If a person's **height** is above a certain *threshold*, we predict 'Male'.



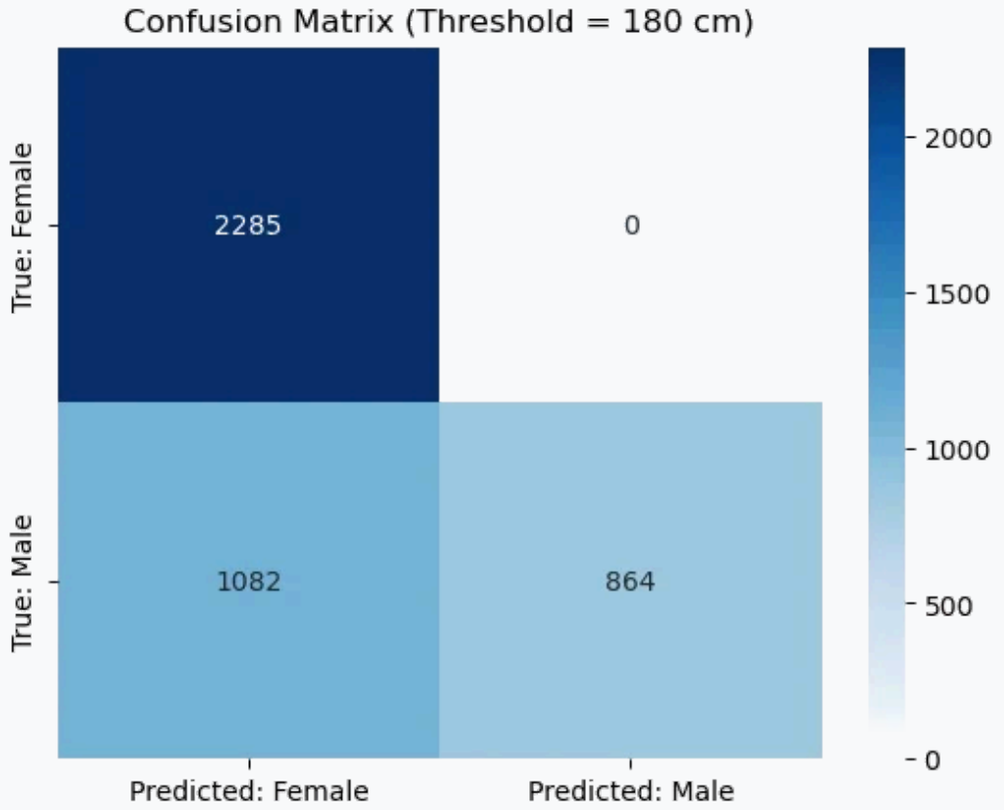
Threshold = 170 cm



	precision	recall	f1-score	support
False	0.88	0.79	0.83	2285
True	0.78	0.87	0.82	1946
accuracy			0.83	4231
macro avg	0.83	0.83	0.83	4231
weighted avg	0.83	0.83	0.83	4231

Accuracy 0.83, and the two classes are treated almost evenly: recall 0.79 on the negatives and 0.87 on the positives.

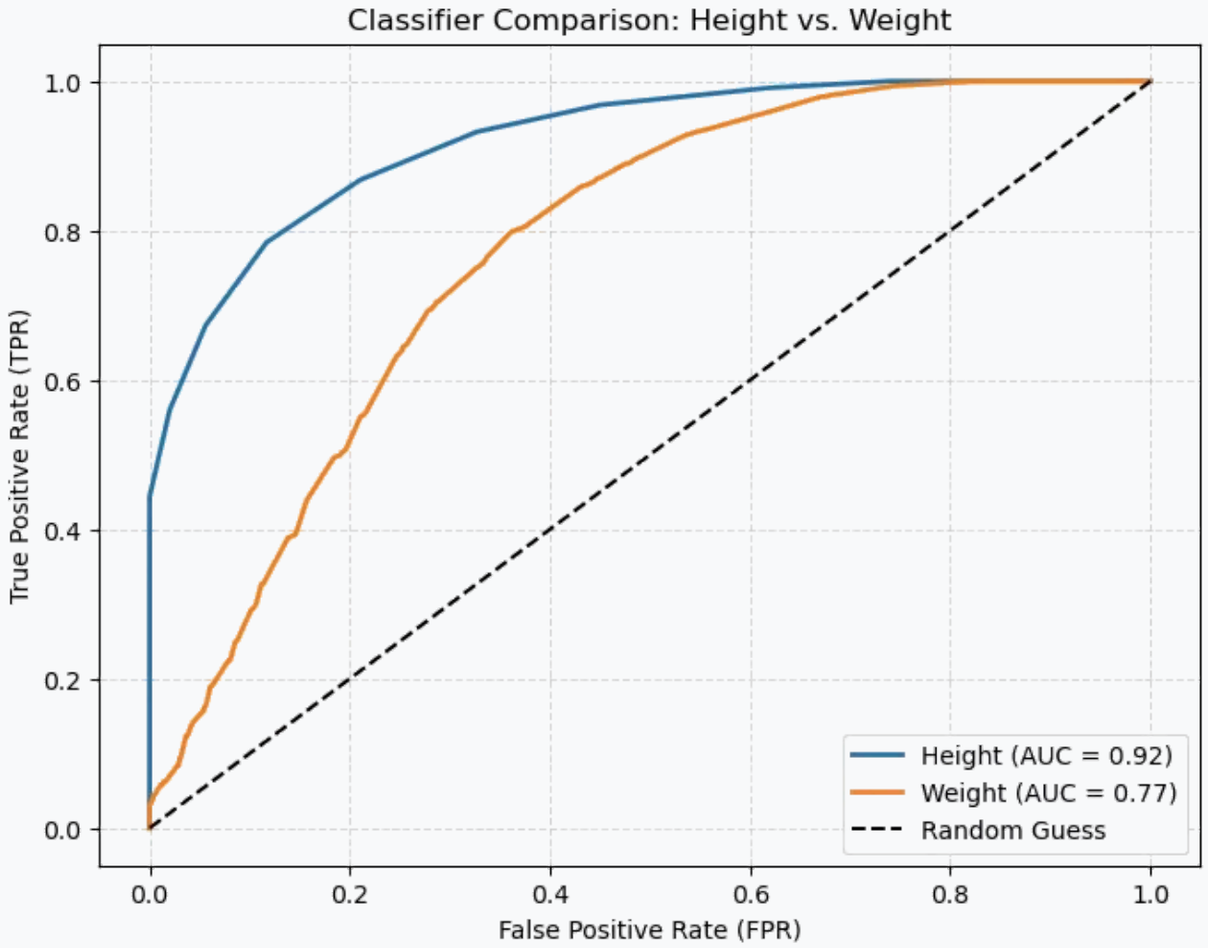
Threshold = 180 cm



	precision	recall	f1-score	support
False	0.68	1.00	0.81	2285
True	1.00	0.44	0.61	1946
accuracy			0.74	4231
macro avg	0.84	0.72	0.71	4231
weighted avg	0.83	0.74	0.72	4231

Move the threshold up and **precision on the positives becomes perfect (1.00) while their recall collapses to 0.44**: no woman is misclassified, and more than half the men are missed. Accuracy drops to 0.74.

ROC Curves for the Two Features



Comparing **height** and **weight** for a threshold-based classifier.

Height is much better than weight, **independent of the threshold**: AUC 0.92 against 0.77.

Finding the Optimal Threshold

The ROC curve shows all possible trade-offs, but we must choose **one** threshold for deployment. The optimal choice depends on the application.

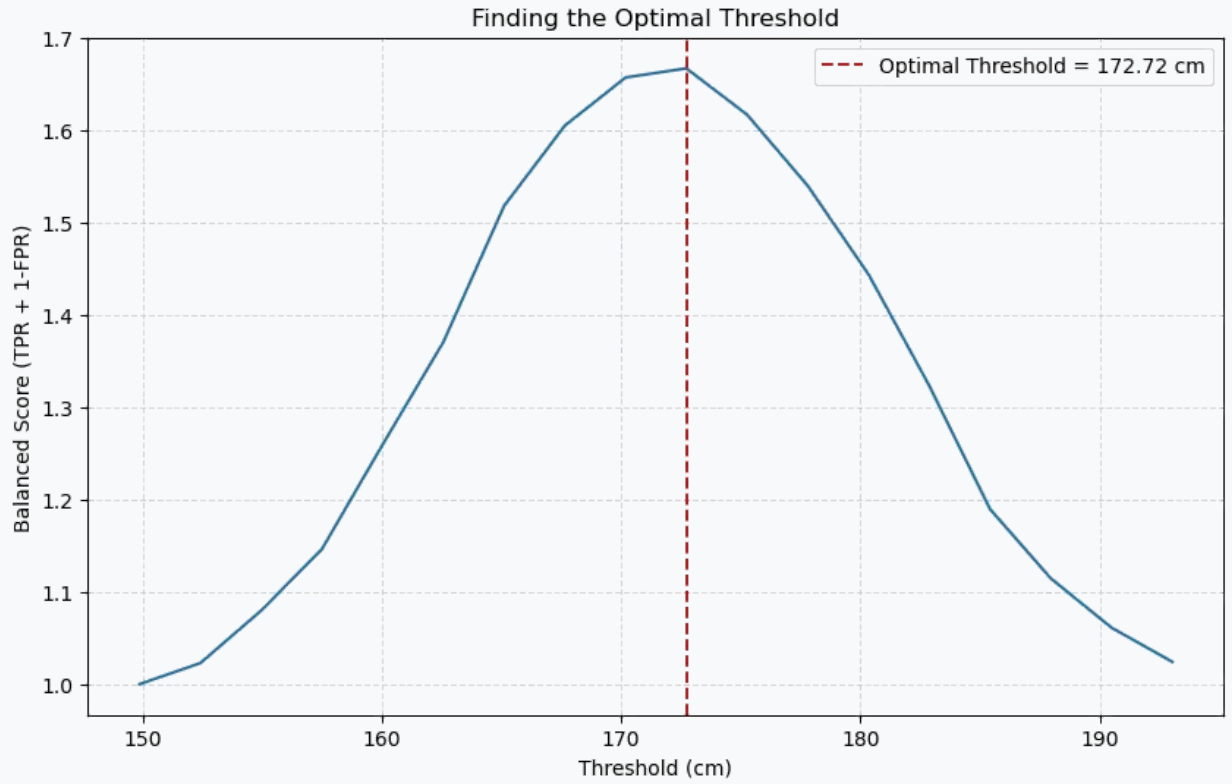
Youden's J statistic

For balanced performance, maximise:

$$J = TPR + (1 - FPR)$$

This finds the threshold closest to the ideal top-left corner of the ROC curve.

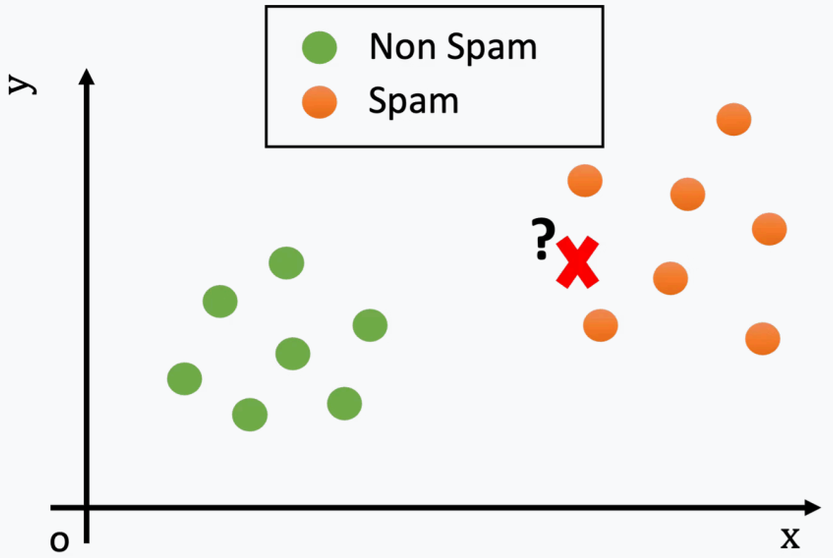
For the height classifier, the optimal threshold is **172.72 cm**, achieving 78% TPR with only 12% FPR — a strong, balanced result.




```
--- Final Model at Optimal Threshold (172.72 cm) ---
True Positive Rate (TPR):  0.78 (We find 78% of all Males)
False Positive Rate (FPR): 0.12 (We mislabel only 12% of Females)

Classification Report at Optimal Threshold:
              precision    recall  f1-score   support
       False       0.83      0.88      0.85      2285
        True       0.85      0.78      0.82      1946
    accuracy                           0.84      4231
```

Nearest Neighbour Algorithm (1-NN)



What's the class of the new (cross) sample?

We can measure how "similar" two examples $\mathbf{x}$ and $\mathbf{x}'$ are using a suitable **distance function** d:

$$d(\mathbf{x}, \mathbf{x}')$$

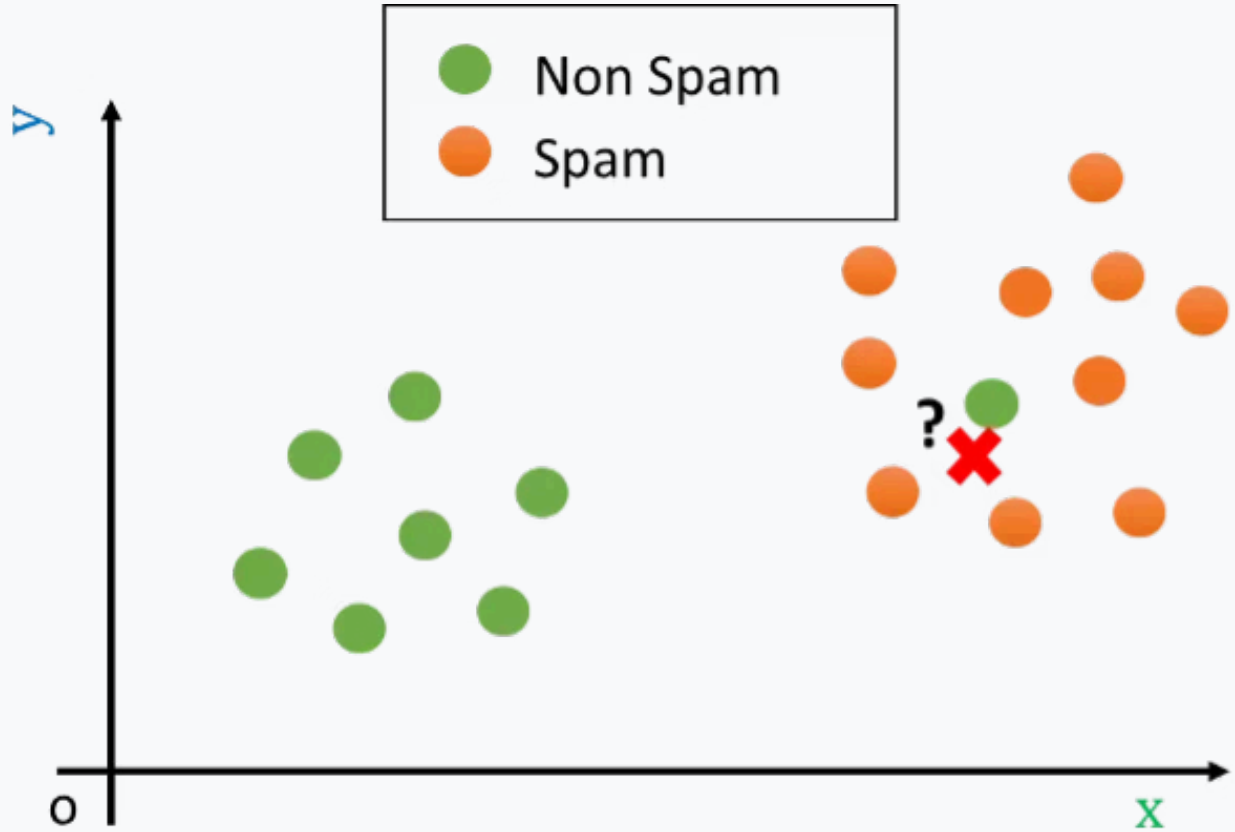
We expect similar examples to have a small distance. A very common choice for d is the **Euclidean distance**:

$$d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_2 = \sqrt{\sum_{i=1}^{N} (x_i - x'_i)^2}$$

We can hence define a classifier which **assigns to a test example $\mathbf{x}'$ the class of the closest example in the training set**:

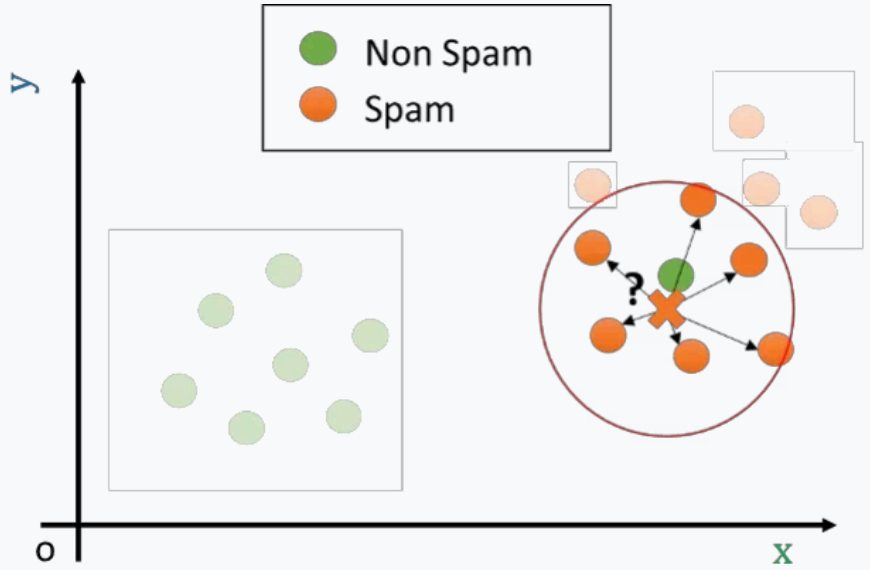
$$h(\mathbf{x}') = \arg_y \min\{d(\mathbf{x}', \mathbf{x}) \mid (\mathbf{x}, y) \in TR\}$$

Outliers and 1-NN



- What if the dataset contains **outliers**?
- Using 1-NN here can bring suboptimal results.
- However, we know that the **second** nearest neighbour would already be a better choice!

K-Nearest Neighbour: Core Intuition



KNN classification relies on a simple principle: **similar examples in feature space likely belong to the same class**.

Measure Distance

Calculate Euclidean distance between test point and all training examples

Find Neighbours

Identify the K closest training examples

Majority Vote

Assign the most common class among the K neighbours

The algorithm assumes nearby points share characteristics.

K-Nearest Neighbour: Addressing 1-NN Limitations

Instead of considering only the single closest point, K-NN evaluates a **neighbourhood** of several points. By considering multiple neighbours, the algorithm becomes more resilient to individual noisy or mislabelled data points.

Defining the K-neighbourhood

Rather than a fixed radius — which can lead to neighbourhoods with varying numbers of points, or even zero in sparse areas — K-NN defines a neighbourhood by the **K closest data points**:

$$N_K(\mathbf{x}') = N(\mathbf{x}', R_K(\mathbf{x}')) \qquad R_K(\mathbf{x}') = \sup\{r : |N(\mathbf{x}', r) \setminus \{\mathbf{x}'\}| \leq K\}$$

K-NN classification algorithm

The K-NN classifier assigns the class that is most frequent among these K neighbours:

$$h(\mathbf{x}') = mode\{y \mid (\mathbf{x}, y) \in N(\mathbf{x}'; TR, K)\}$$

- **The role of K.** K is a **hyperparameter** of the algorithm. Choosing an optimal K is crucial and is typically done through **cross-validation** or by evaluating performance on a **validation set**. A 1-NN algorithm is simply the special case $K = 1$.

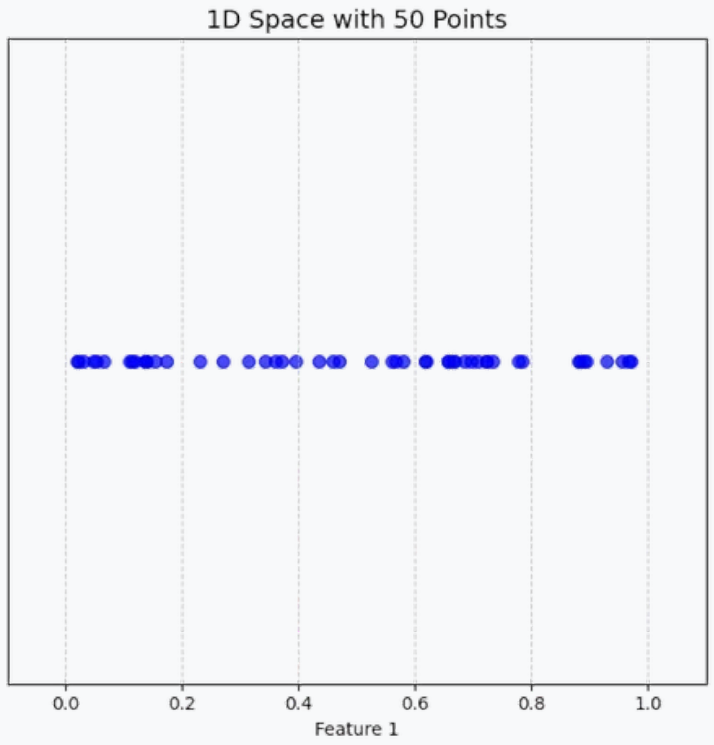
The Curse of Dimensionality

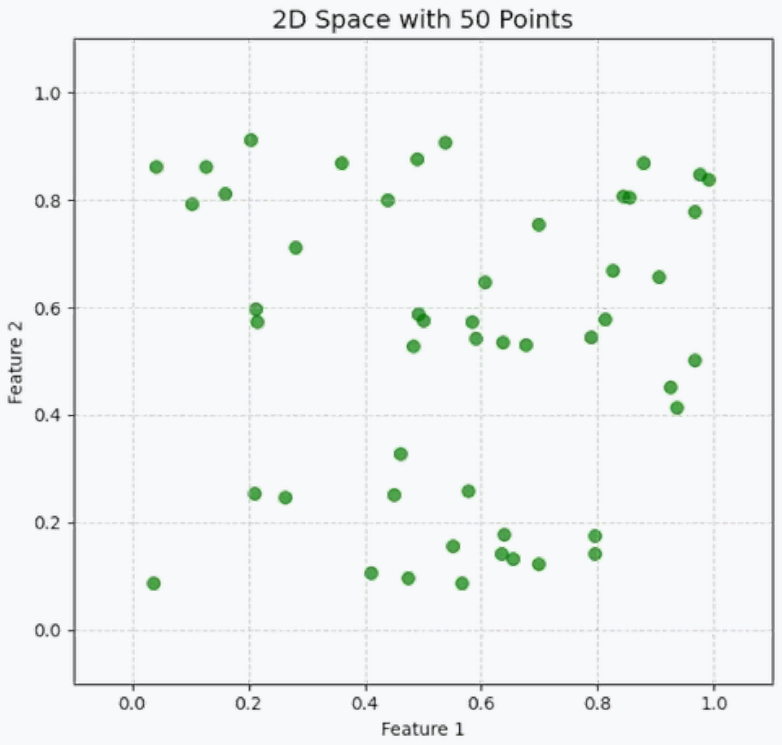
KNN's fatal flaw

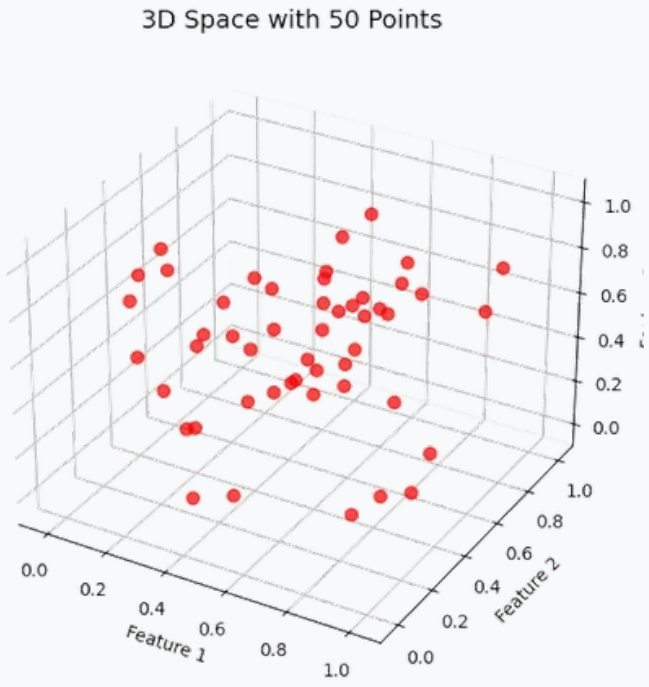
As features increase, data becomes **exponentially sparse** and distance becomes meaningless. This is KNN's greatest weakness.

The "empty space" problem: 2D against 3D

The same fifty points, in one, two and three dimensions. Nothing was added or removed — the space around each point simply grew.







50

Data Points

Same number across dimensions

1000×

Volume Growth

Space expands exponentially

~0

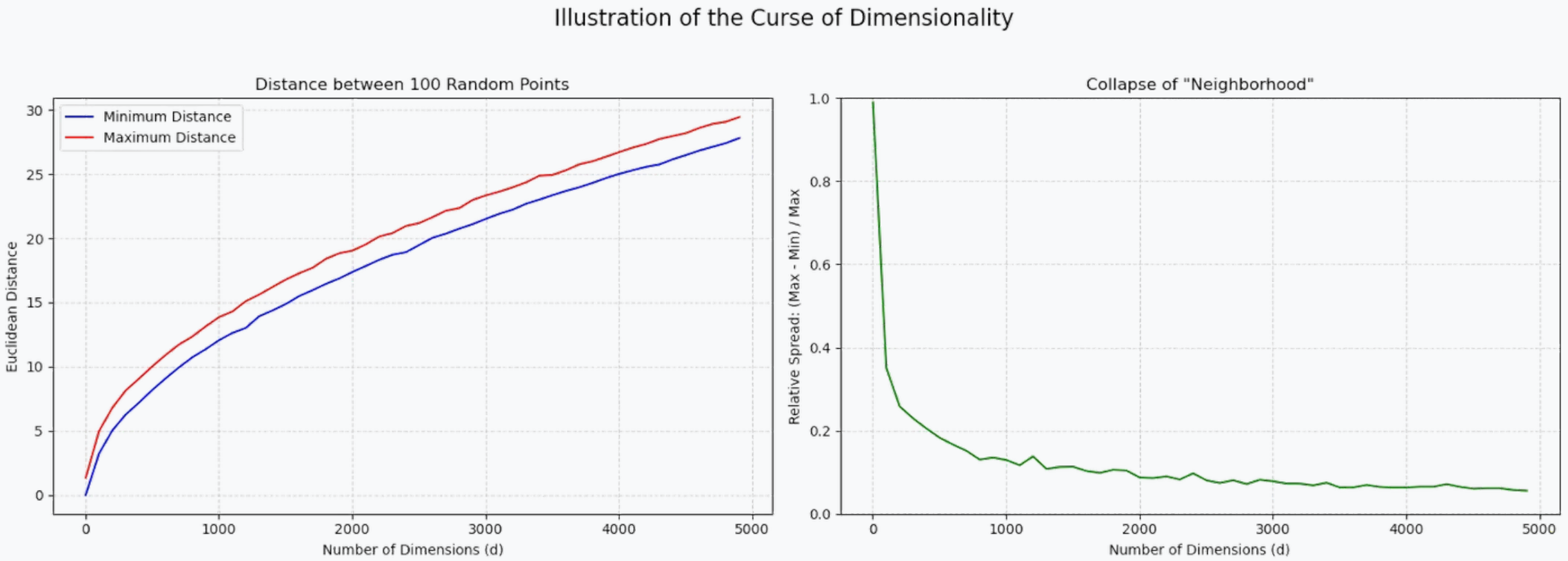
Distance Contrast

Min and max distances converge

The Curse of Dimensionality — cont.

The impact on KNN

In high dimensions, your "nearest" neighbour is almost as far as your "farthest" neighbour. **The concept of "nearness" collapses, and KNN predictions become random.**



- **Practical limit:** KNN works excellently for 2–10 features but fails for high-dimensional data (images: 10 000+ features, text: 5 000+ features) without dimensionality reduction.

KNN: A Discriminative Classifier

Understanding the distinction between **generative** and **discriminative** classifiers clarifies how models like K-Nearest Neighbours approach classification.

Generative classifiers

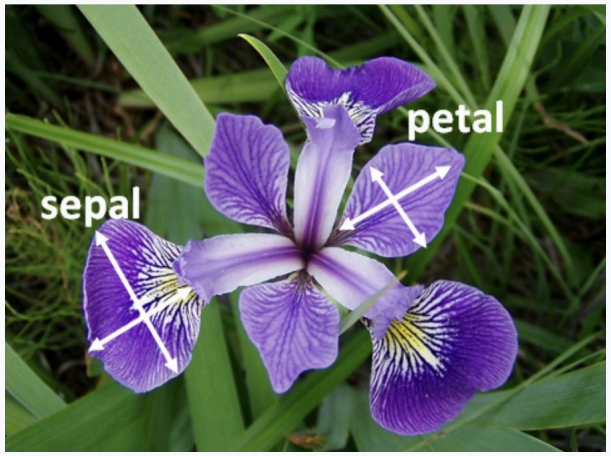
- **What they do:** learn the underlying probability distribution of *each class*. They build a "story" for what each class looks like.
- **The question they answer:** "What does Class A look like?"
- **How they classify:** use knowledge of class distributions (e.g., via Bayes' theorem) to compute $P(y|X)$.
- **Example:** Naive Bayes.

Discriminative classifiers

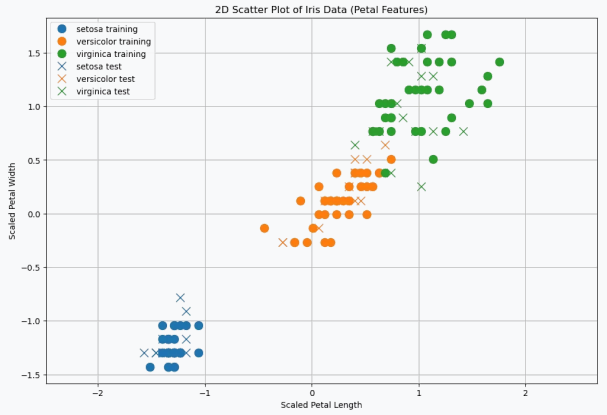
- **What they do:** *ignore* the underlying distribution of individual classes. They only focus on finding the **decision boundary** that separates one class from another.
- **The question they answer:** "What is the *difference* between Class A and Class B?"
- **How they classify:** directly learn $h(x)$ or the boundary $P(y|X)$.
- **Example:** K-Nearest Neighbours and Logistic Regression.

KNN on the Iris Dataset

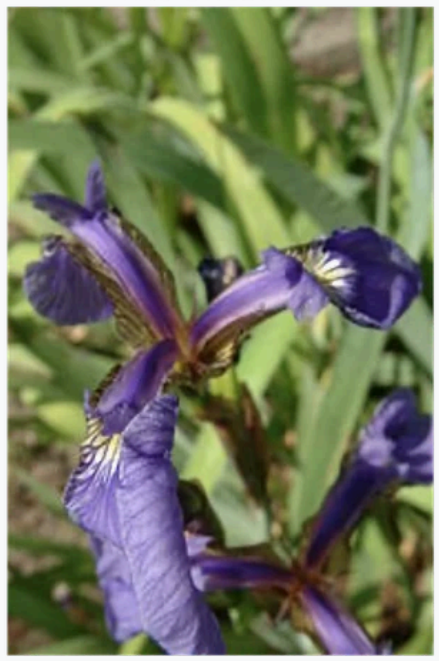
The Fisher Iris dataset is the "hello world" of classification: **150 flowers from 3 species**, measured by 4 features (sepal and petal length and width).



	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
5	6	5.4	3.9	1.7	0.4	Iris-setosa
6	7	4.6	3.4	1.4	0.3	Iris-setosa
7	8	5.0	3.4	1.5	0.2	Iris-setosa
8	9	4.4	2.9	1.4	0.2	Iris-setosa
9	10	4.9	3.1	1.5	0.1	Iris-setosa



Iris Setosa



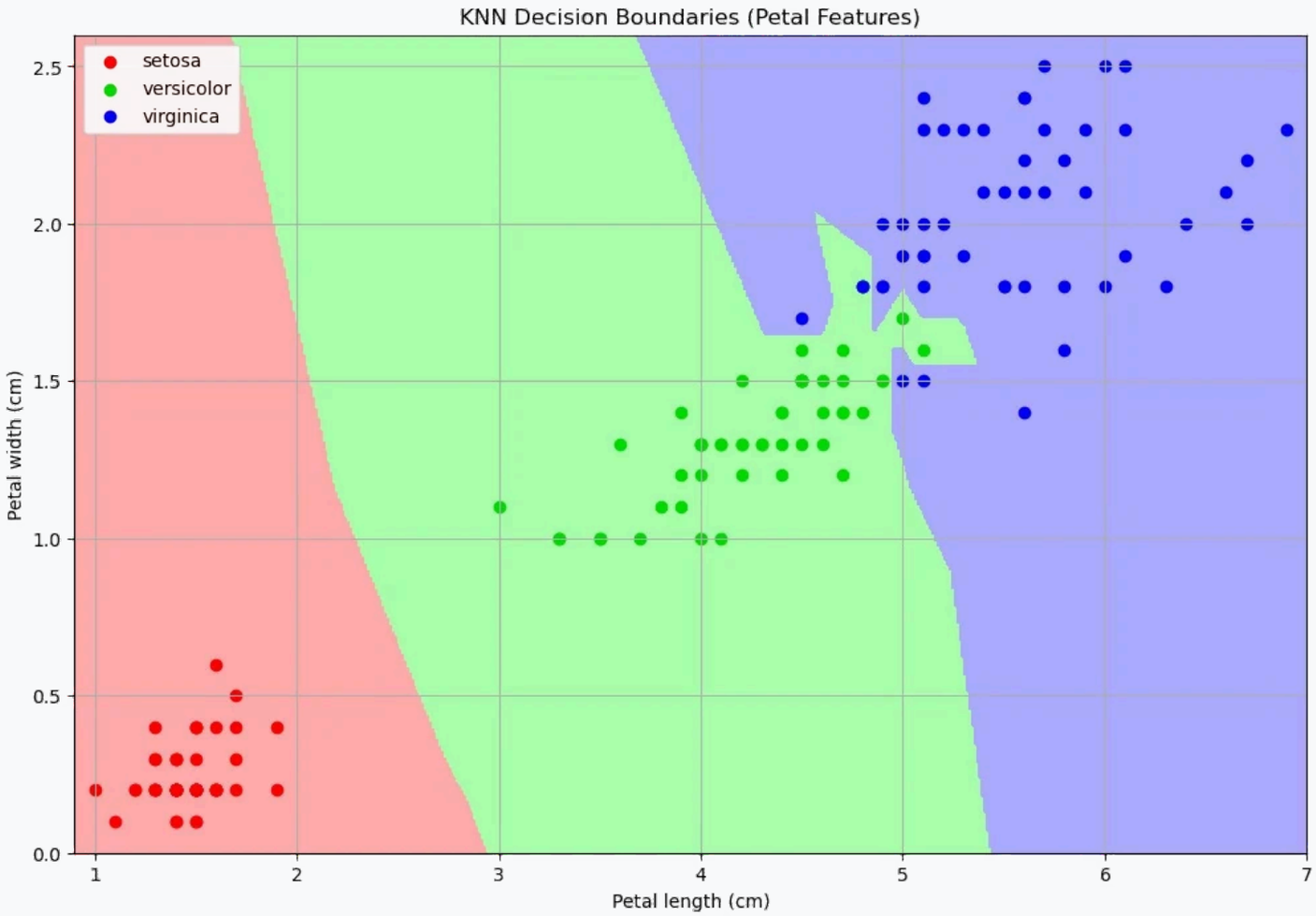
Iris Versicolor



Iris Virginica

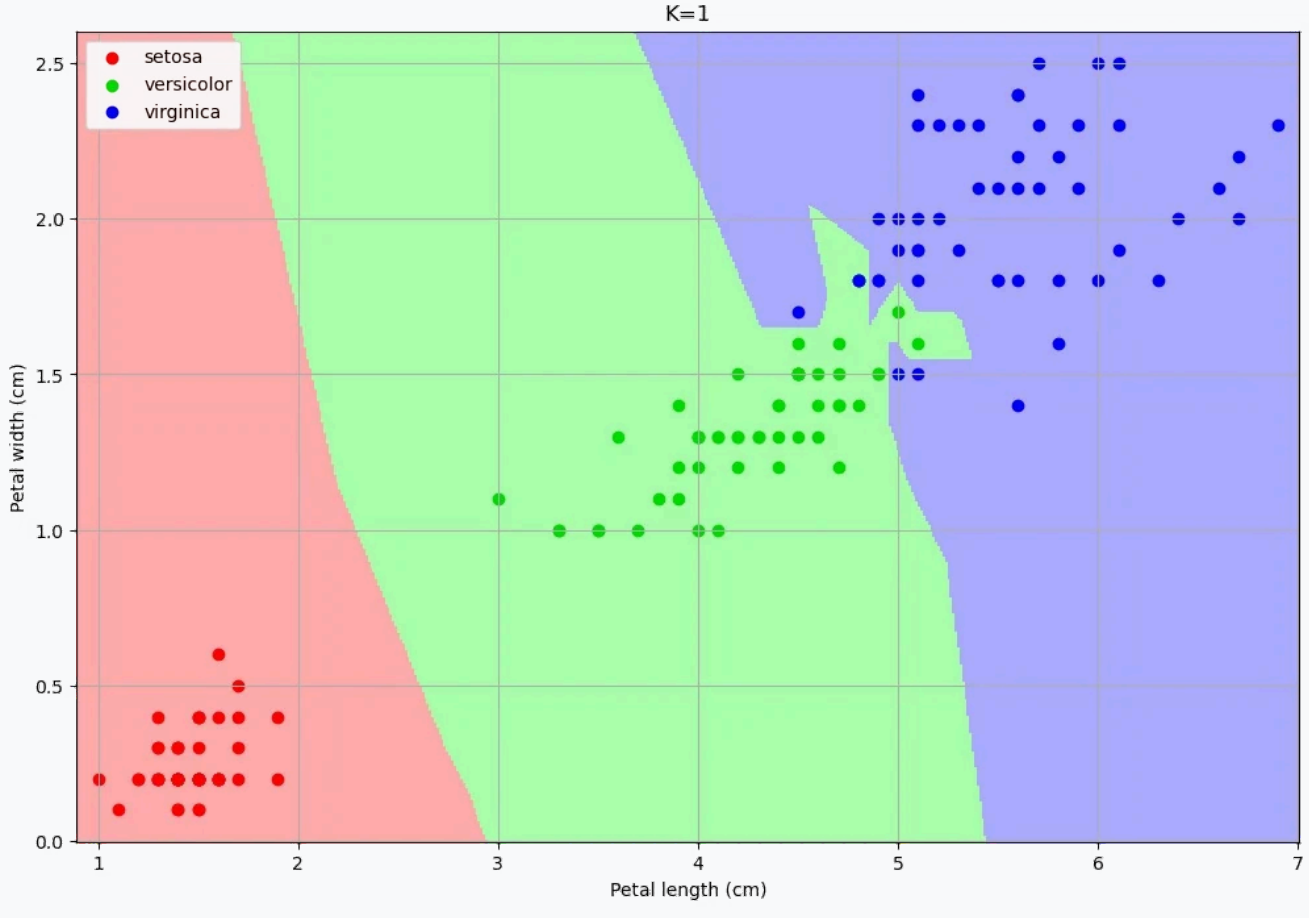


KNN on the Iris Dataset — cont.



Using only **two features** — petal length and petal width — we can visualise the 2-D decision space and understand how KNN creates classification boundaries.

The Decision Boundary



A classifier f assigns a class to each input $\mathbf{x}$. Since $\mathbf{x}$ can be seen as a geometrical point in the n-dimensional space, it is interesting to see how the classification function works **on a portion of the representation space**.

This is done by generating a **classification map** or **decision boundary**, obtained by computing the label the classifier would assign to a dense grid of data points.

Points of different colours are the data; regions of different colours are how that point in space would be classified.

The Impact of K on Decision Boundaries

The hyperparameter K controls the **bias-variance trade-off**, dramatically affecting how the algorithm partitions feature space.

1

Small K

High variance, low bias. Over-segments space, sensitive to outliers. **Overfitting.**

2

Medium K

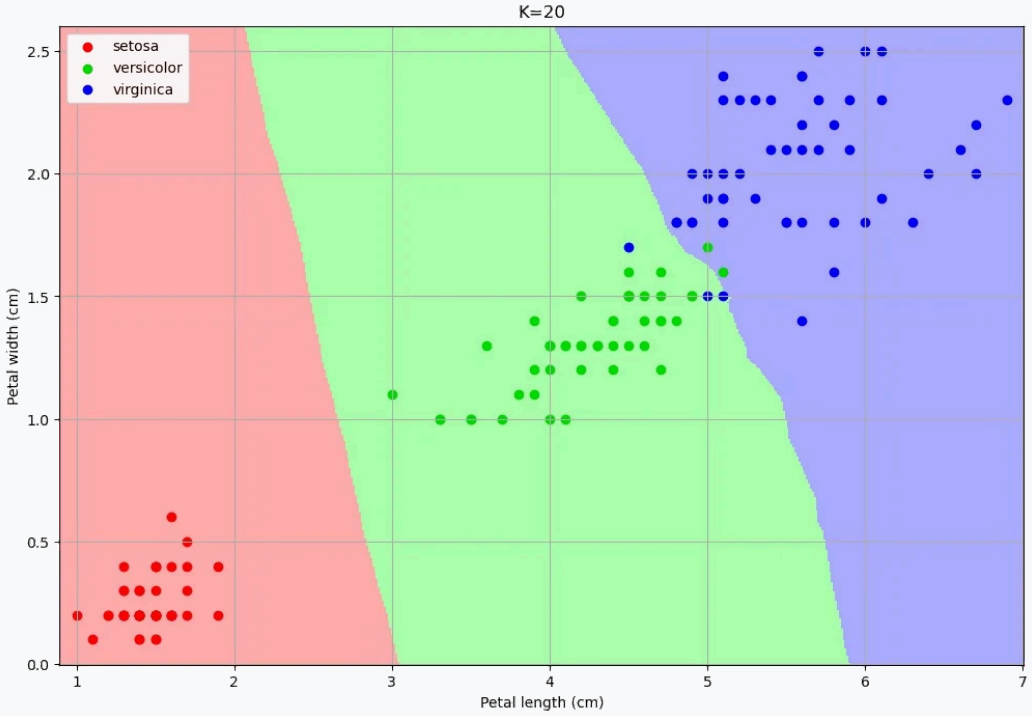
Balanced. Smooth regions while respecting local patterns. **Optimal range.**

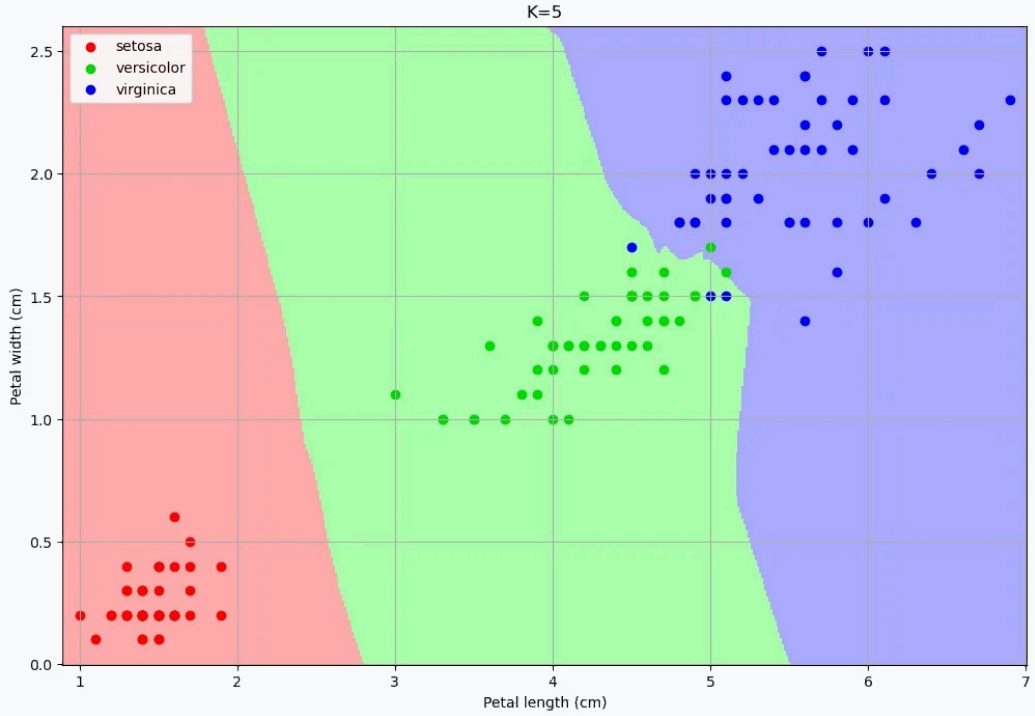
3

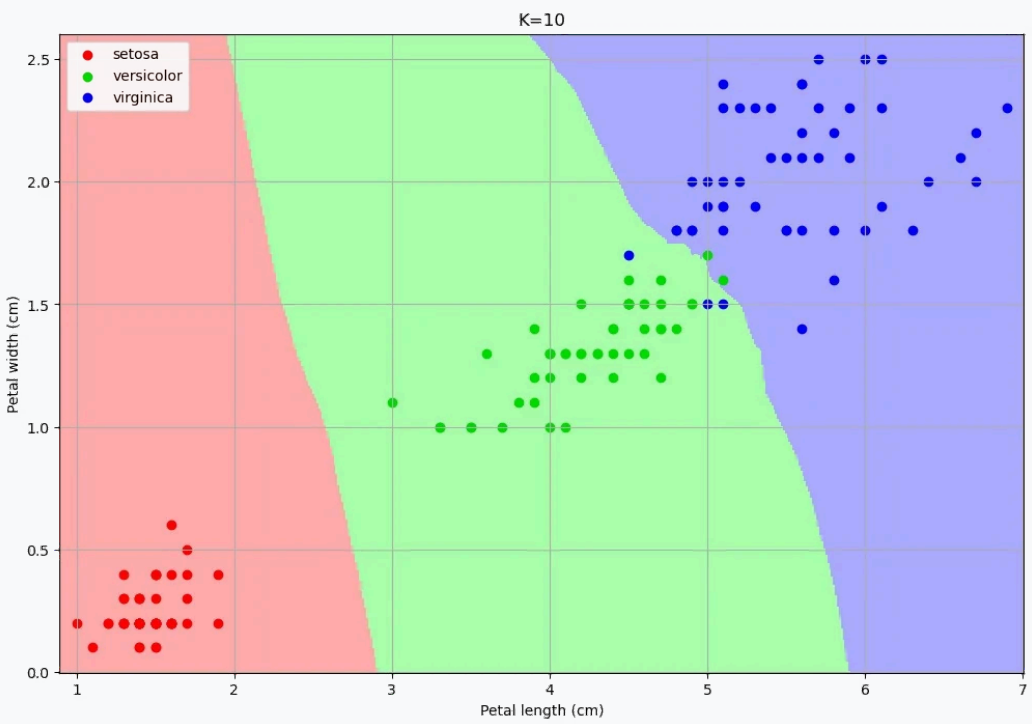
Large K

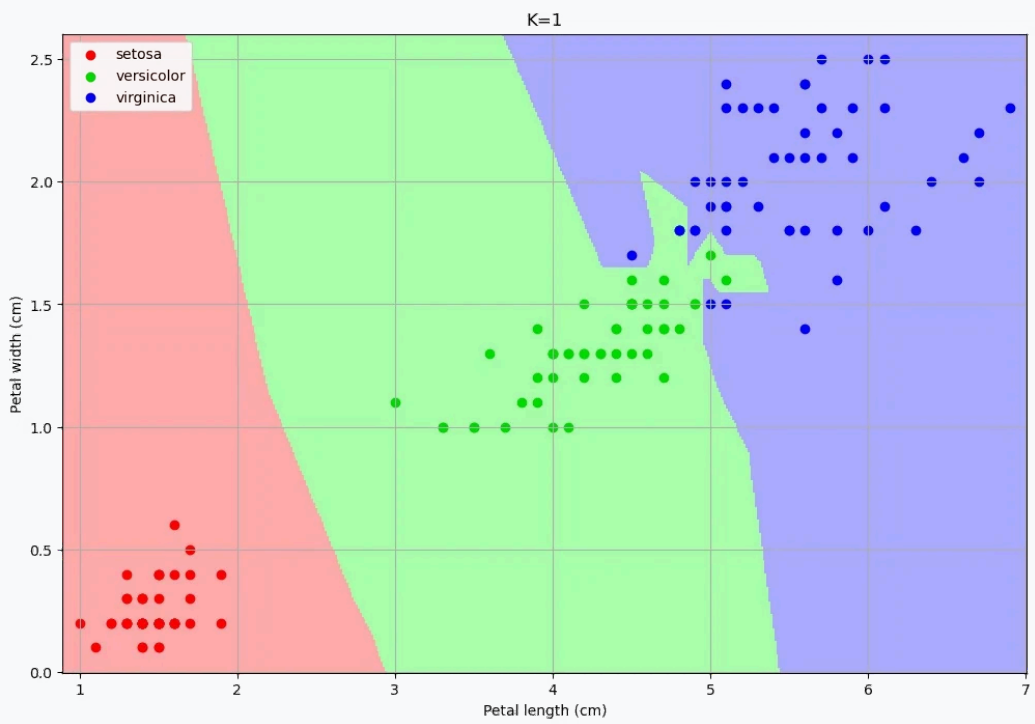
Low variance, high bias. Ignores local structure. **Underfitting.**

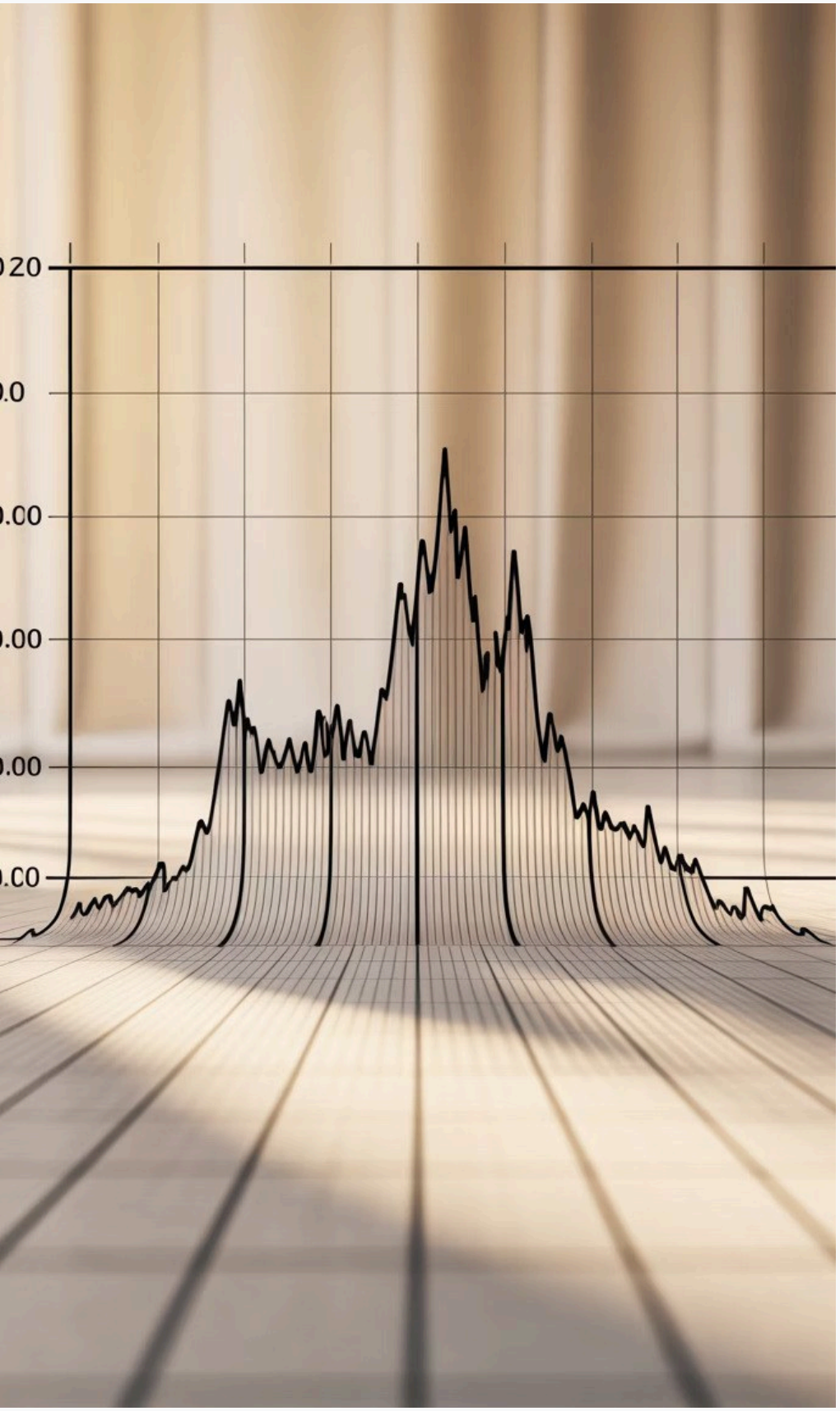
Visualising the bias-variance trade-off: overfit against underfit











Finding Optimal K with Cross-Validation

Cross-validation systematically tests different K values on held-out folds to find the optimal hyperparameter **without touching the test set**.

01

Split Data

Create train/test split. Lock away the test set.

02

Cross-Validate

For each K, perform 5-fold CV on training data. Record mean accuracy.

03

Select Best K

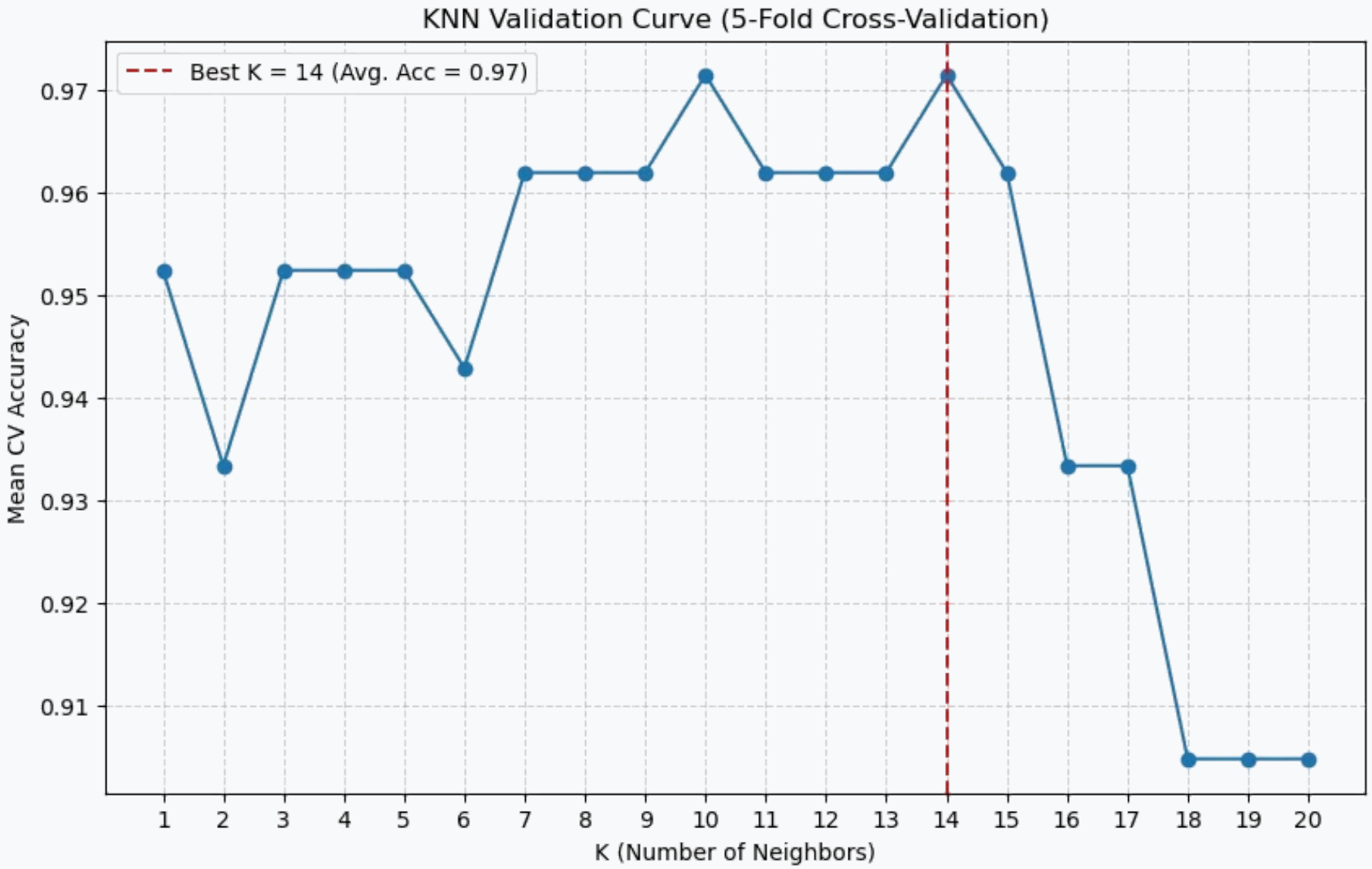
Choose K with highest CV accuracy — **K = 14** in our example.

04

Final Evaluation

Train on the full training set with the best K. Evaluate once on the test set: **95.56% accuracy**.

Finding the Best K by Cross-Validation



The curve is not smooth: with 105 training flowers, a single reassigned point moves the accuracy by one percentage point. **K = 14 wins, but everything from 7 to 15 is within noise of it** — which is worth saying out loud when reporting a tuned hyperparameter.

KNN for Regression

KNN extends naturally to regression by predicting the **mean** of the K neighbours instead of the mode. The algorithm is identical except for the final aggregation step.

Classification vs regression

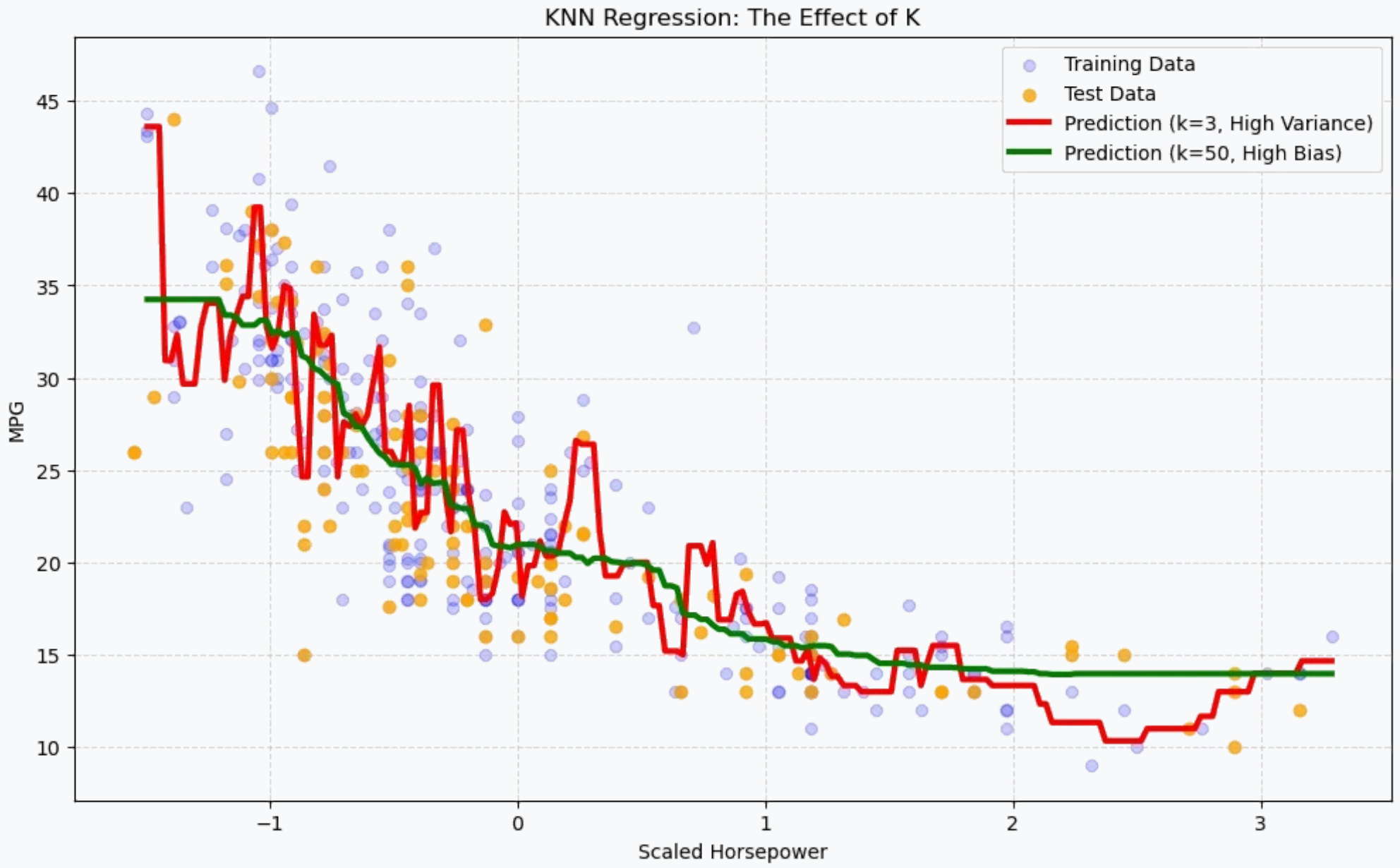
- **Classification:** majority vote of K neighbours' labels
- **Regression:** average of K neighbours' values

Same distance calculation, same neighbour selection, different aggregation.

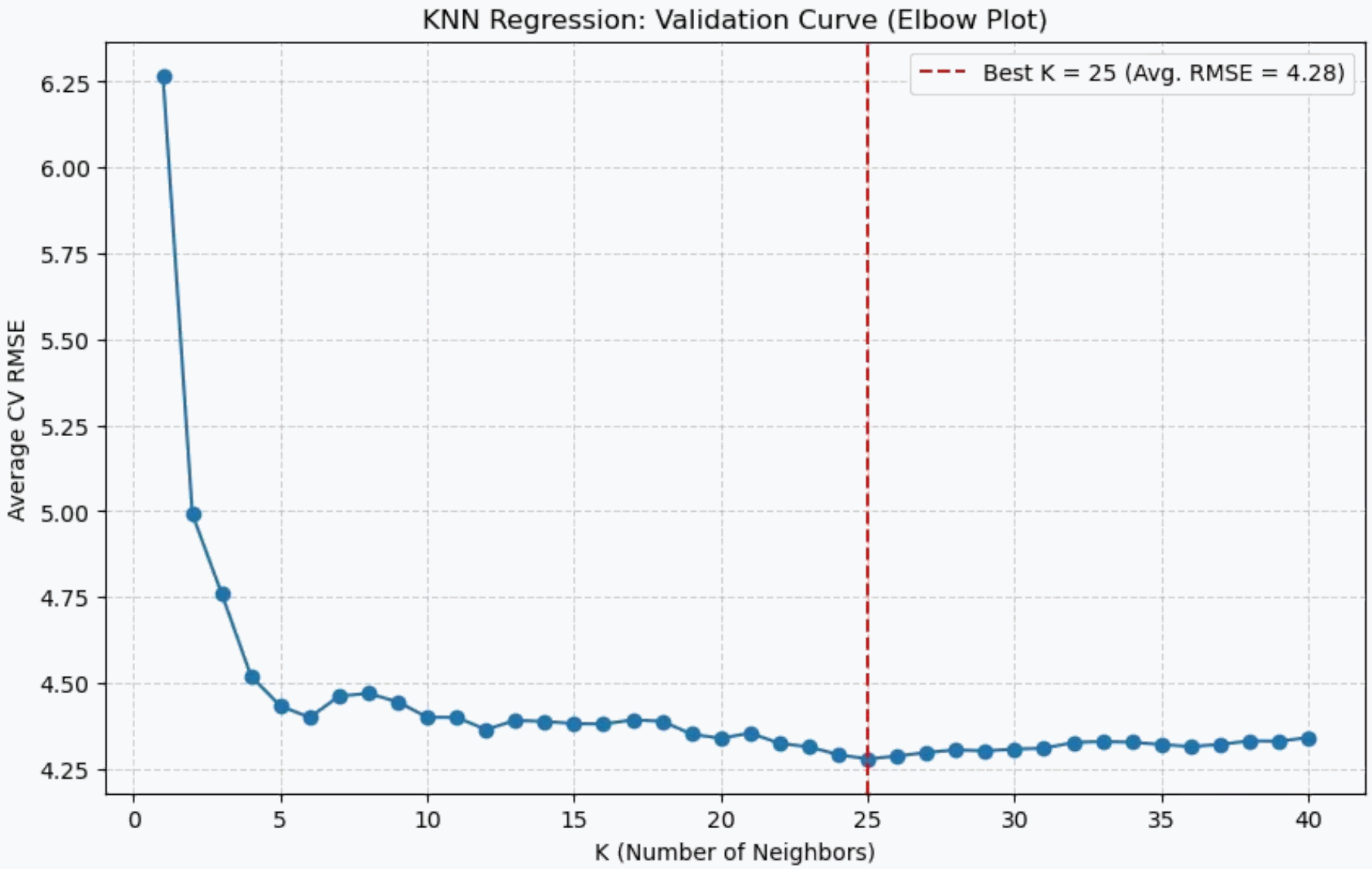
MPG prediction example

Predicting fuel efficiency from horsepower using KNN regression. Optimal **K = 25** found via cross-validation.

Final test RMSE: 4.29 mpg



Finding the Best K for Regression



The elbow plot for regression: RMSE falls steeply to about K = 5, then flattens. **K = 25 is the minimum, but the curve is nearly flat from 10 onwards** — a reminder that the "best" hyperparameter is often a plateau, not a peak.

KNN in Python



```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, cross_val_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

```
Training samples: 105
Test samples:     45

Model Accuracy (k=5): 91.11%

--- Confusion Matrix ---
[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]
```

```
Cross-validation complete. The best K is 14.

--- Final Model Evaluation ---
Final Test Set Accuracy (with K=14): 95.56%

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.88      1.00      0.94        15
   virginica       1.00      0.87      0.93        15
```

Tuning K by cross-validation moved the test accuracy from **91.1% to 95.6%** — two flowers, on a test set of 45. Small numbers; report them with the size of the set they came from.

Multiclass, and Where It Comes From

Chapter 10 fitted **multinomial logistic regression**: one baseline class, $K-1$ sets of coefficients, each read as a log-odds ratio against that baseline. As a *classifier* the same model needs no reading of coefficients — only the class with the highest predicted probability.

$$h(\mathbf{x}) = \arg\max_k P(Y = k|\mathbf{x})$$

The boundary between two classes is where their probabilities are equal, $P(Y = i|\mathbf{x}) = P(Y = j|\mathbf{x})$, which simplifies to the equality of their linear scores:

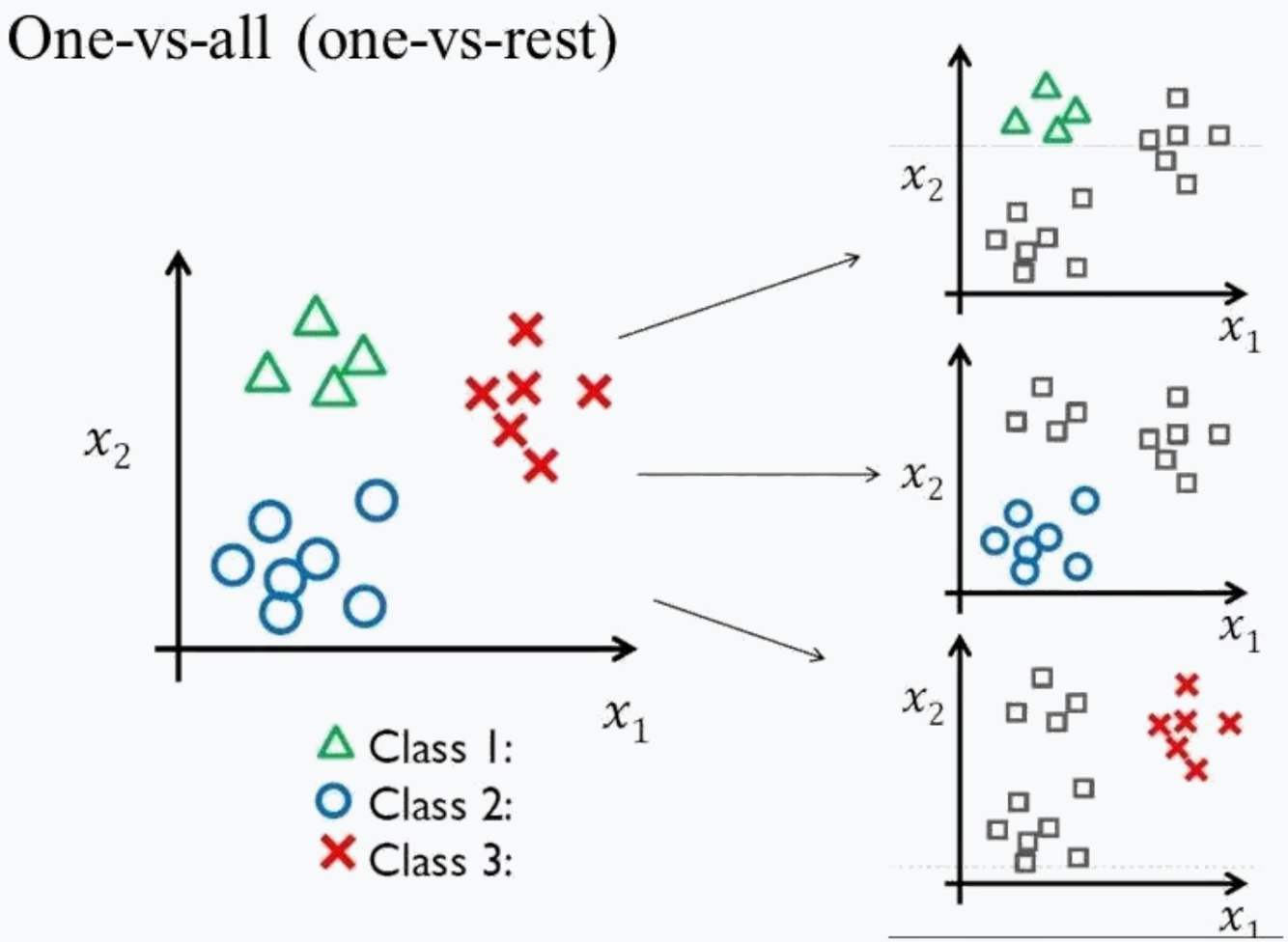
$$\beta_i^T \mathbf{x} = \beta_j^T \mathbf{x} \Rightarrow (\beta_i - \beta_j)^T \mathbf{x} = 0$$

a **hyperplane** — a straight line in two dimensions. The model partitions the feature space into regions, one per class, always separated by linear boundaries.

- `scikit-learn` writes this model without a baseline, giving every class its own coefficients — the **softmax** parametrisation noted in chapter 10. It is the same model, overparametrised; we do not use it to interpret coefficients here, and **it is not examined**.

There is a second route to multiclass, which does not change the model at all: **one-vs-rest**.

From Binary to Multi-Class: One-vs-Rest



One-vs-Rest turns **any binary classification model into a multi-class model**. Also known as "one vs all".

It turns one K-class problem into K separate binary problems. For this we need a binary classifier that outputs a confidence score — a probability — so we can find the most likely class.

One-vs-Rest — cont.

01

Deconstruct the problem

Train K independent binary classifiers $h_k(\mathbf{x})$. Each is trained to distinguish class k from all other classes (the "rest"), by relabelling the training set: **positive** = instances of class k, **negative** = instances of any class $j \neq k$.

02

Classify a new instance

Feed $\mathbf{x}$ to all K classifiers. Each outputs a confidence score, typically a probability:

$$s_k(\mathbf{x}) = P(y = k|\mathbf{x})$$

03

Select the predicted class

Choose the class whose classifier is most confident:

$$\hat{y} = \arg \max_{k \in \{1,\ldots,K\}} s_k(\mathbf{x})$$

Multiclass Logistic Regression in Python

In lecture 9 we used `statsmodels` to build logistic models for **inference**: p-values and log-odds. Here the goal is **predictive accuracy**, so the workflow changes:

01

Train/test split

For an honest, unbiased evaluation

02

Pipeline with `StandardScaler`

Logistic regression is sensitive to feature scales, especially when regularised

03

`GridSearchCV` over `C`

`C` is the **inverse** of the regularisation strength λ: small `C` is strong regularisation

04

Evaluate once on the test set

Accuracy, classification report, confusion matrix

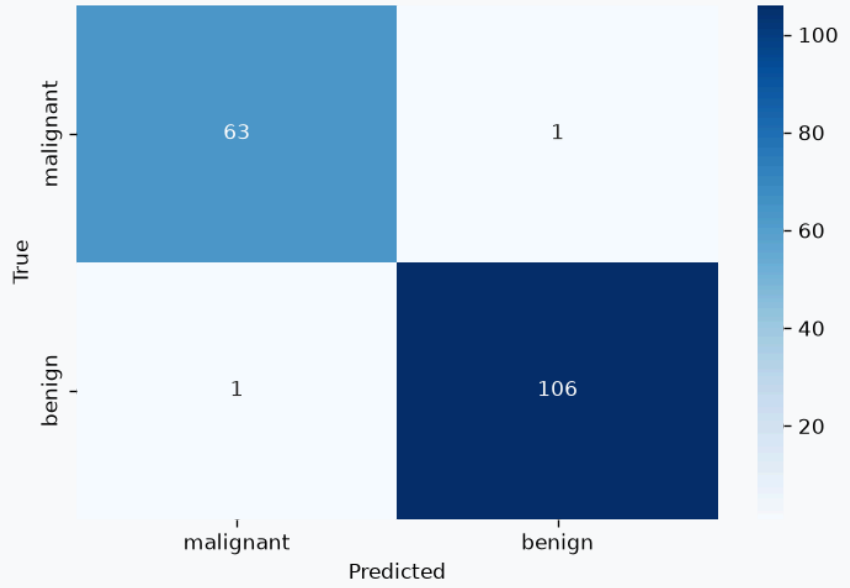
```
Best 'C' found by CV: 1.0
Best CV accuracy:     0.9799

--- Final Model Evaluation (Test Set) ---
Overall Accuracy: 98.83%

              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        64
      benign       0.99      0.99      0.99       107
```

On the Breast Cancer Wisconsin data — 30 features, two classes — the tuned pipeline misses **two of 171** test cases.



Multinomial against One-vs-Rest, Measured

Both strategies, tuned the same way on the Iris data:

```
Running Grid Search for the multinomial model...
Best 'C' for Multinomial: 1.0
Best CV Accuracy: 0.9810

Running Grid Search for One-vs-Rest (OvR)...
Best 'C' for OvR: 10.0
Best CV Accuracy: 0.9714
```

Model	Test accuracy
Multinomial	**0.911**
One-vs-Rest (OvR)	0.867

- The multinomial model wins here, and it usually does when the classes are mutually exclusive: it optimises all K boundaries **together**, while OvR fits each classifier in ignorance of the others. OvR earns its place when the underlying binary model has no multiclass form — which is why `scikit-learn` keeps both.

Conclusions and Next Steps



We Have Explored:

The classification task

Classification measures, and averaging them

ROC curves and choosing a threshold

The threshold-based classifier

K-NN, and the curse of dimensionality

K-NN for regression

Multiclass: multinomial and one-vs-rest

The scikit-learn workflow for classifiers

- In the next lecture we look at generative classifiers: QDA, LDA and naive Bayes.

References

- Evaluation measures for classification: **en.wikipedia.org/wiki/Precision_and_recall**
- Nearest neighbour: Section 2.5.2 of [1]

[1] Bishop, Christopher M. *Pattern recognition and machine learning*. Springer, 2006. **Link**

- Lecture notes: **Chapter 15 — Classification**